

CSGO Market Study

Pablo Santana Acosta, Óscar González Pérez, Luis Wandelmer Peña

2025-12-22

Indice

- Introducción
- Términos
- Objetivos del trabajo
- Fuentes de datos utilizadas
- Análisis de los datos
- Conclusiones

Introducción

En esta investigación trataremos de dar luz a un mercado muy grande pero que está oculto detrás de un videojuego, queremos ver que tanto se comporta como uno usual y como de controlado está. Usaremos los métodos aprendidos durante el año y aquellos que nos enseñaron en primer año en la clase de economía empresarial.

Términos

Durante este trabajo usaremos términos asociados a este mercado como “skins”, “armas” o “loot boxes”. Por lo que antes de comenzar queremos explicarlos para evitar confusión.

Primero y antes de todo, el mercado de CSGO (también conocido como Counter Strike Global Offensive o Counter Strike 2 después de la actualización del 27 de Septiembre de 2023) es un mercado que intercambia dinero (en el caso de nuestra base de datos Yuanes Chinos) por agentes cosméticos dentro del juego que después pueden ser re-vendidos entre usuarios.

Mientras los usuarios juegan tienen una probabilidad de conseguir una “loot box” que con una “key” que tiene un precio fijo durante todo el tiempo de 2.50\$ en EEUU y 2.10€ en EU (excepuando loot boxes especiales que se consiguen en eventos como competiciones que requieres llaves más caras para abrirlas). Con esta “loot box” y “key” que les llamaremos caja y llave intercambiabilmente, pueden conseguir un cosmético aleatorio dentro de lo que cada caja (que varían cada temporada) traiga.

Existen muchas cajas que un usuario normal puede conseguir y por lo tanto muchos cosméticos que pueda sacar, pero las cajas están trucadas para favorecer que salgan cosméticos más baratos que caros, por lo que es bien conocido en la comunidad que “nunca ganas dinero abriendo cajas” (aunque nosotros lo investigaremos nuevamente a ver cuanto ganas/pierdes de media).

Ahora, ya sabemos de donde salen los cosméticos, porqué crece el mercado y conceptos como caja y llave, ahora ataquemos el concepto de “skin” y “armas”. Realmente el concepto correcto es cosméticos, pero como los cosméticos son principalmente usados en las armas del juego hay veces que los usamos intercambiablemente. Ahora, nuestro conjunto de datos no incluye las keys pero si las cajas, por lo que es importante recordar que cuando hablamos de “skins”, “armas”, “items” no es más que elementos que puedes comprar del mercado para re-venderlos, abrirlas si son cajas o usarlas si son cosméticos.

Por último tampoco mencionaremos skins hechas a mano. Dicho de manera simple, mezclando skins con stickers y cambiando el nombre o inclusive subiendo estadísticas del arma individual como “kill counter” puedes crear un arma única. Lo ignoramos porque estas suelen ser objetos de coleccionismo que rara vez circula por el mercado y a lo mucho tienen 2-3 ventas con precios super altos (especialmente si la hizo una persona famosa), y tampoco podríamos añadirlas porque la base de datos no las cuenta.

Objetivos del trabajo

- Estudiar el mercado
- Predecir su crecimiento/decrecimiento
- Analizar qué fluctuaciones se puede esperar (volatilidad del mercado)
- *Analizar las probabilidades, precios y rentabilidad asociadas a las “loot boxes”.

*En nuestra base de datos no tenemos los precios de las keys ni de los items que dan, por lo que cuando lleguemos a ese apartado tendremos que buscar las estadísticas que “Valve” aporta públicamente (items que pueden tocar, probabilidades y precio de la key asociada). Todo esto será citado como fuentes extras en los anexos a medida que aportamos datos.

Fuentes de datos utilizadas

En este trabajo usaremos un repertorio público “Archive of CS2 Item’s Price History from Buff.163” que almacena durante el periodo (aproximadamente) de 26.07.2021 - 19.01.2024 en varias definiciones el mercado online de CSGO (y su consecuente actualización a CS2) desde el punto de vista de una de las páginas de terceros más populares para hacer compras y ventas llamada “buff.163.com”.

Notas importantes de esta fuente de datos son:

- No tiene datos sobre keys, pero tiene sobre cases.
- No tiene datos de demanda antes de 25/01/2023 (tiene 0)
- Las medias para cualquier dataframe diferente a raw se calculan como: `aggregate = dataPoints.filter(d => d > 0).length == 0 ? 0 : dataPoints.filter(d => d > 0).median()`.
- Los datos se guardan como centiyuanes (para que sean números enteros). Por lo que tuvimos que pasarlo a Yuanes normales (CN¥) dividiendo entre 100.
- Hay ~300 items que no fueron medidos y se pueden encontrar cuales fueron mirando un .txt dentro del repertorio aquí.
- No contiene skins hechas a mano (con stickers) o con nombres especiales, por ejemplo una skin “firmada” por una persona famosa, ya que cada una es única y no es representativo de lo que la skin original valdría.

Tratamiento de datos

Aunque no es el enfoque de este trabajo, al haber tomado una cantidad de trabajo substancial consideramos apropiado hablar sobre como pasamos los datos de su formato comprimido a uno más manejable.

Todo esto lo trata el main y en caso que se quiera actualizar con nuevos datos este fichero llama a los scripts que harán este trabajo posible. Este trabajo fue posible a causa de nuestra habilidad para pasar json comprimidos a un formato .rds comprimido que permita almacenamiento y carga/descarga rápida. De no haber podido haber hecho esto los tiempo de carga hubieran sido tan elevados que lo hubiera hecho impráctico.

Aunque no vayamos a entrar en detalle en el código invito a cualquier lector a que mire los scripts para que pueda ver el trabajo que tomó (tanto en tiempo de computación como en código) para transformar un .json a los dataframes que damos por sentado en este trabajo.

Análisis de los datos

Limpiar el dataframe

Cargamos las librerías que usaremos y el dataframe.

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.1      v stringr   1.5.2
## v ggplot2    4.0.0      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.1.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(dplyr)
library(tidyr)
library(purrr)
library(ggplot2)
library(lubridate)
library(nortest)
library(scales)
```

```
##
## Adjuntando el paquete: 'scales'
##
## The following object is masked from 'package:purrr':
##
##     discard
##
## The following object is masked from 'package:readr':
##
##     col_factor
```

```
data_month <- readRDS("./out/data_monthly.rds")
```

Primero de todo debemos tratar el principal problema con el dataframe, que usa el sistema de Steam donde si no hay oferta el precio es 0. Esto es problemático porque hay skins que se venden rara vez pero por mucho dinero y no es representativo para el valor del mercado que porque no se venda su valor es 0. Para solventar esto haremos que reserve el valor del último que tengan (si era 0 porque es un item nuevo y nunca se ha comprado y vendido seguirá estando y tendremos que sesgarlos cuando haga falta) pero esto permite una representación real de un mercado.

```
# Helper: parsea una celda a (oferta, precio) numéricos o missing
.parse_cell <- function(cell) {
  # missing: NULL real, "NULL", NA, length 0
  if (is.null(cell) || length(cell) == 0 ||
      (length(cell) == 1 && (is.na(cell) || identical(cell, "NULL")))) {
    return(list(missing = TRUE, oferta = NA_real_, precio = NA_real_))
  }

  # Si viene como vector/list con nombres (oferta, precio)
  if ((is.atomic(cell) || is.list(cell)) && length(cell) >= 2) {
    nm <- names(cell)
    if (!is.null(nm) && all(c("oferta", "precio") %in% nm)) {
      return(list(
        missing = FALSE,
        oferta = as.numeric(cell[["oferta"]]),
        precio = as.numeric(cell[["precio"]])
      ))
    }
  }

  # Si viene sin nombres pero con dos valores (oferta, precio)
  if (length(cell) == 2) {
    return(list(
      missing = FALSE,
      oferta = as.numeric(cell[[1]]),
      precio = as.numeric(cell[[2]])
    ))
  }

  # Si viene como string "0.00, 0.27"
  if (is.character(cell) && length(cell) == 1) {
    parts <- strsplit(cell, ",", fixed = TRUE)[[1]]
    if (length(parts) >= 2) {
      return(list(
        missing = FALSE,
        oferta = as.numeric(trimws(parts[1])),
        precio = as.numeric(trimws(parts[2]))
      ))
    }
  }

  # Fallback: si hay algún formato raro
  warning("Formato de celda no reconocido; se deja como missing. str(cell): ",
        paste(capture.output(str(cell)), collapse = " "))
  list(missing = TRUE, oferta = NA_real_, precio = NA_real_)
}
```

```

}

fix_zero_prices_locf <- function(df, id_col = "nombre") {
  month_cols <- setdiff(names(df), id_col)
  month_cols <- month_cols[order(as.Date(paste0(month_cols, "-01")))]

  # Asegura list-cols (para poder guardar NULL y pares oferta/precio)
  for (col in month_cols) {
    if (!is.list(df[[col]])) df[[col]] <- as.list(df[[col]])
  }

  # Normaliza: cada celda -> NULL o c(oferta=..., precio=...) numérico
  df[month_cols] <- lapply(df[month_cols], function(col) {
    lapply(col, function(cell) {
      p <- .parse_cell(cell)
      if (p$missing) return(NULL)
      c(oferta = p$oferta, precio = p$precio)
    })
  })

  # Extrae matrices numéricas (rápido para aplicar LOCF por fila)
  oferta <- sapply(df[month_cols], function(col)
    vapply(col, function(cell) if (is.null(cell)) NA_real_ else unname(cell[["oferta"]]), numeric(1))
  )
  precio <- sapply(df[month_cols], function(col)
    vapply(col, function(cell) if (is.null(cell)) NA_real_ else unname(cell[["precio"]]), numeric(1))
  )

  # Corrige por fila: si oferta==0 & precio==0 => precio = último precio > 0 anterior
  for (i in seq_len(nrow(df))) {
    last_pos <- NA_real_
    for (j in seq_along(month_cols)) {
      o <- oferta[i, j]
      p <- precio[i, j]

      if (!is.na(p) && p > 0) last_pos <- p

      if (!is.na(o) && o == 0 && !is.na(p) && p == 0 && !is.na(last_pos)) {
        precio[i, j] <- last_pos
      }
    }
  }

  # Vuelca precios al df (manteniendo NULL intactos)
  for (j in seq_along(month_cols)) {
    colname <- month_cols[j]
    col <- df[[colname]]
    newp <- precio[, j]

    for (i in seq_along(col)) {
      cell <- col[[i]]
      if (is.null(cell)) next
      cell[["precio"]] <- newp[[i]]
    }
  }
}

```

```

      col[[i]] <- cell
    }
    df[[colname]] <- col
  }

  df
}

data_month_adjusted <- fix_zero_prices_locf(data_month)

```

Min y Max del Dataframe

Simplemente hacemos un formato que permita hacer la búsqueda de forma más rápida y lo mostramos.

```

df_long <- data_month_adjusted %>%
  pivot_longer(
    cols = -nombre,
    names_to = "fecha",
    values_to = "valor"
  ) %>%
  mutate(
    oferta = map_dbl(valor, \(x) {
      if (is.null(x)) return(NA_real_)
      as.numeric(x["oferta"])
    }),
    precio = map_dbl(valor, \(x) {
      if (is.null(x)) return(NA_real_)
      as.numeric(x["precio"])
    })
  ) %>%
  select(-valor)

df_long %>%
  summarise(
    min_precio           = min(precio, na.rm = TRUE),
    max_precio           = max(precio, na.rm = TRUE),
    min_precio_real      = min(precio[precio > 0], na.rm = TRUE),
    min_oferta           = min(oferta, na.rm = TRUE),
    min_oferta_mayor_que_0 = min(oferta[oferta > 0], na.rm = TRUE),
    max_oferta           = max(oferta, na.rm = TRUE)
  )

```

```

## # A tibble: 1 x 6
##   min_precio max_precio min_precio_real min_oferta min_oferta_mayor_que_0
##   <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
## 1         0  3499990         0.02         0         1
## # i 1 more variable: max_oferta <dbl>

```

Como podemos ver, podemos esperar un rango real esperable de:

- Precio: [0.02, 3499990]

- Demanda: [1, 274841]

Para el caso en el que no usemos casos ideales podemos esperar ofertas y densidades de 0 (las máximas se mantienen igual).

Gráficos de Métricas

Generamos un dataframe con los datos de interés.

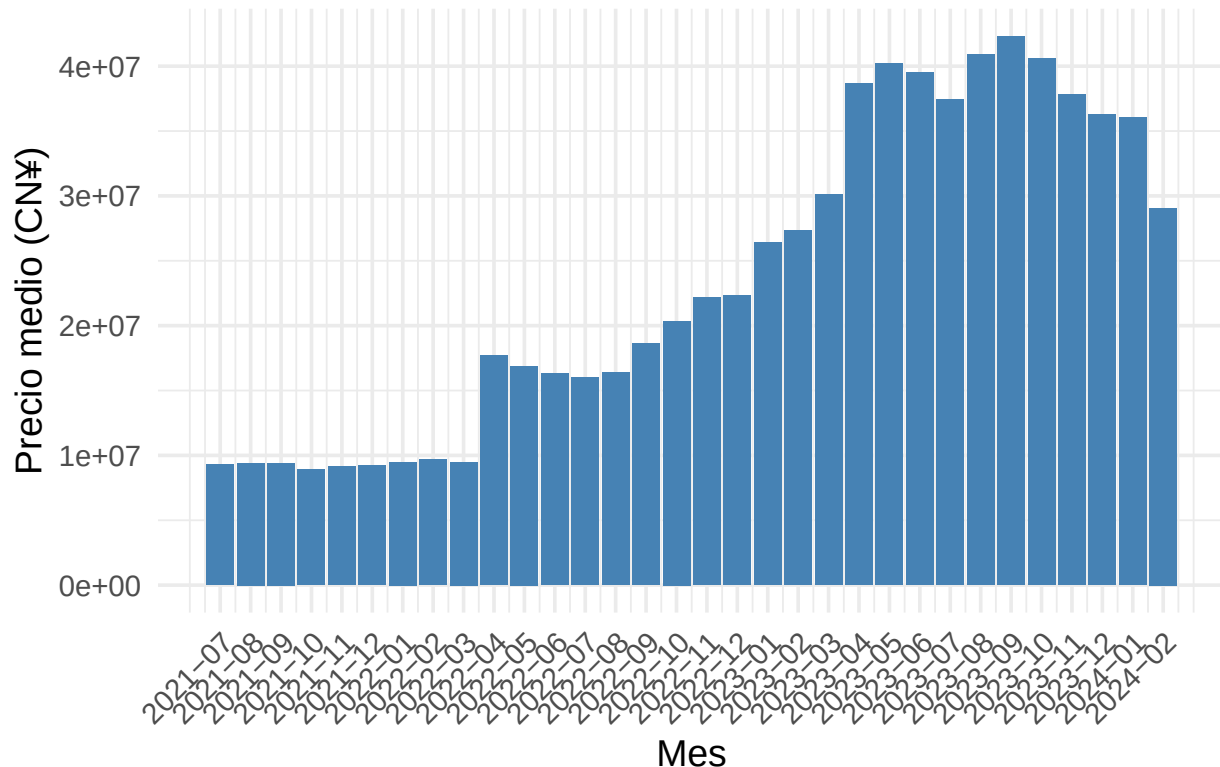
```
mercado_media <- df_long %>%
  filter(!(oferta == 0 & precio == 0)) %>%
  group_by(fecha) %>%
  summarise(
    media_precio = mean(precio, na.rm = TRUE),
    media_demanda = mean(oferta, na.rm = TRUE),

    suma_precios = sum(precio, na.rm = TRUE),
    suma_demanda = sum(oferta, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(fecha) %>%
  mutate(mes_num = row_number())
```

Y con ese ahora podemos crear los gráficos.

```
ggplot(mercado_media, aes(x = as.numeric(factor(fecha)), y = suma_precios)) +
  geom_col(aes(x = as.numeric(factor(fecha))), fill = "steelblue") +
  scale_x_continuous(
    breaks = seq_along(mercado_media$fecha),
    labels = mercado_media$fecha
  ) +
  theme_minimal(base_size = 14) +
  labs(
    title = "Evolución del precio mensual del mercado",
    x = "Mes",
    y = "Precio medio (CN¥)"
  ) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Evolución del precio mensual del mercado



Aquí podemos ver la evolución del mercado mensualmente, es decir que es la suma de todos los precios durante ese mes. No usamos la formula económica que es la demanda * precio porque tendríamos que sesgar más de la mitad de nuestros datos ya que los datos no recuerdan la demanda para fechas antes de febrero de 2023.

De igual manera, para el usuario medio este es el crecimiento que le importa, el de los precios y como podemos ver hemos visto un crecimiento constante con una subida cuando salió CS2 en septiembre de 2023 y el inicio de un descenso al final del dataframe, indicando una bajada.

Esta bajada se debe en parte al reciente uso de “bot farms” para conseguir items usando IAs remotas, de esa manera tienen varias copias del juego corriendo a la vez en un ordenador, generando “drops” a medida que estas juegan entre ellas para después venderá. Esto a su vez se volvió especialmente rentable en China donde su economía permite tales gastos eléctricos para recuperarlo con la venta ilegal de items en mercados Europeos y Estado Unidense.

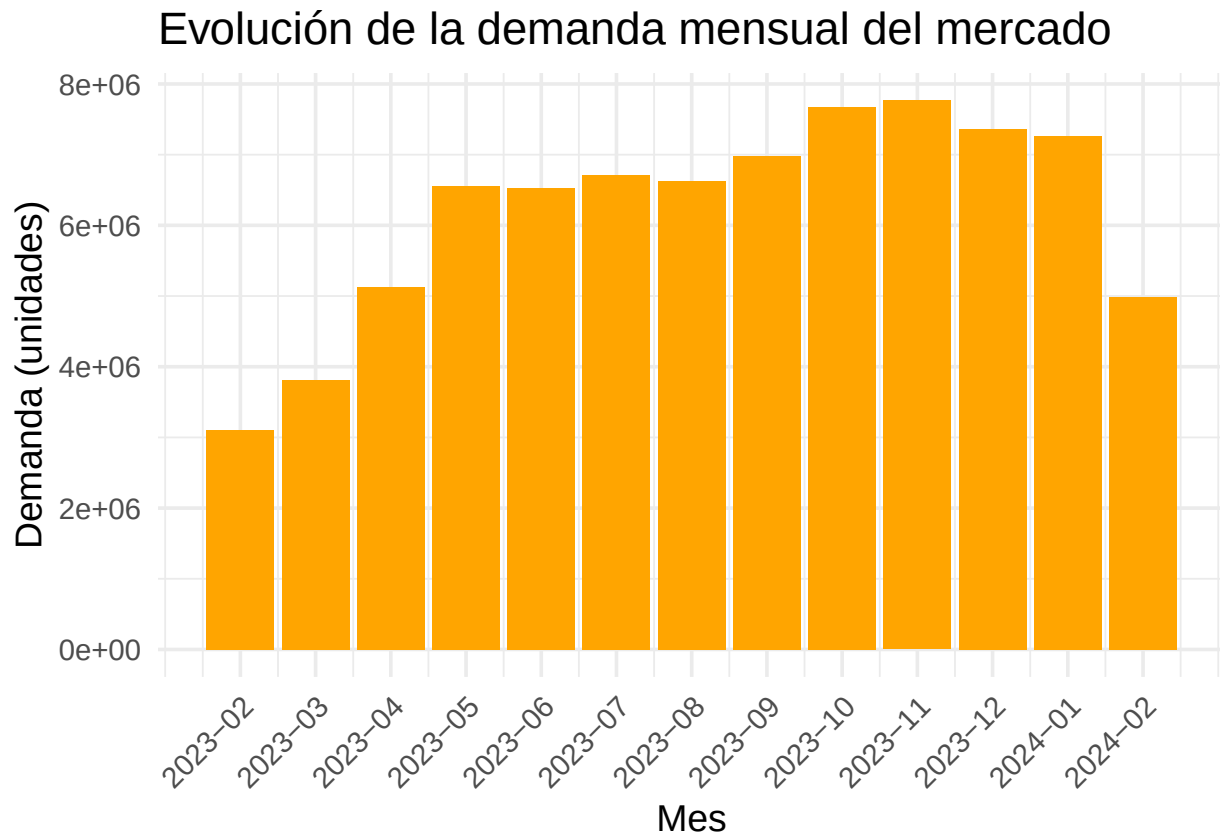
*Este dato fue logrado gracias a jugadores profesionales como “SSpookyce” que generosamente se ofreció a darnos contextos sobre algunos eventos que encontramos en este trabajo.

```
mercado_media_demanda <- mercado_media %>%
  filter(fecha >= "2023-02")

ggplot(mercado_media_demanda, aes(x = mes_num, y = suma_demanda)) +
  geom_col(fill = "orange") +
  scale_x_continuous(
    breaks = mercado_media_demanda$mes_num,
    labels = mercado_media_demanda$fecha
  ) +
  theme_minimal(base_size = 14) +
```

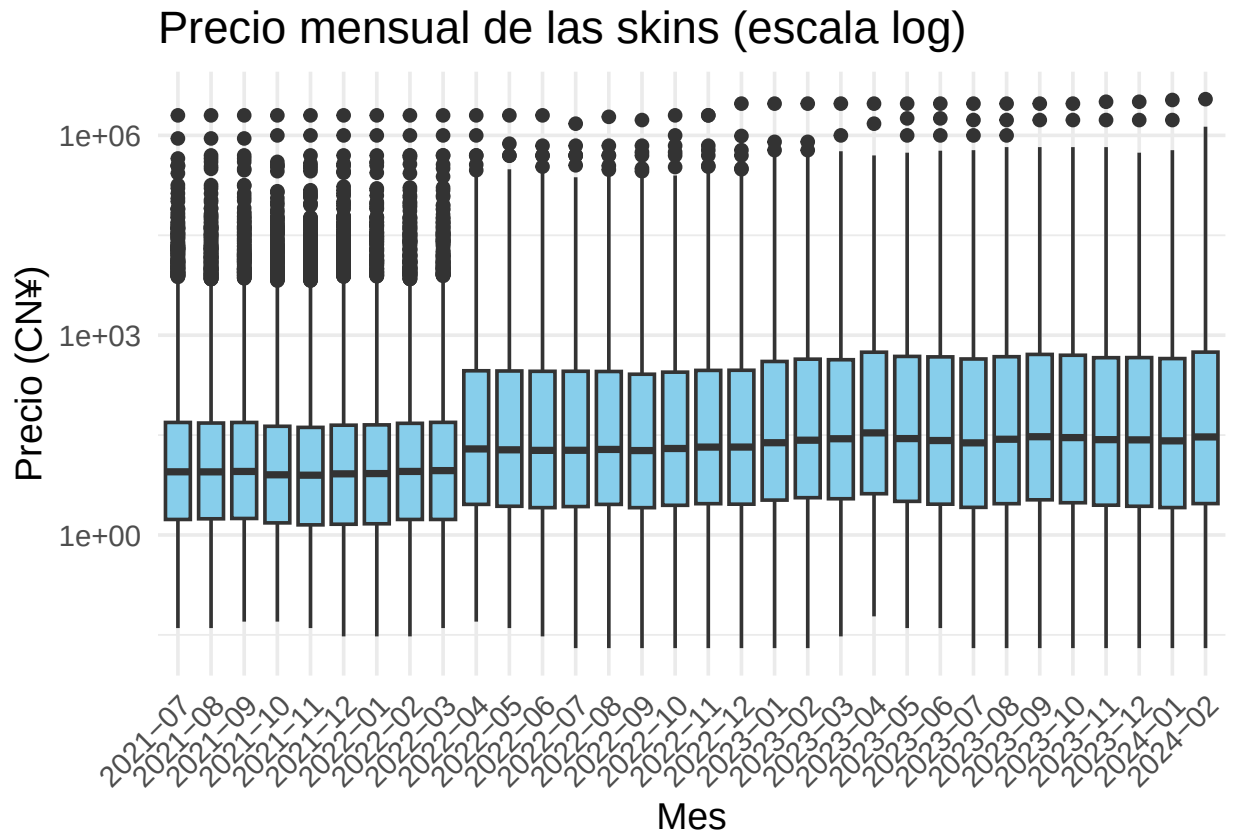


```
labs(
  title = "Evolución de la demanda mensual del mercado",
  x = "Mes",
  y = "Demanda (unidades)"
) +
theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



Aquí se ve la oferta de items y como esta creció con el tiempo, también se ve la bajada en 2024, realmente esta subió pero nuestros datos están incompletos en ese mes, por lo que esto explica porque no lo pudimos explicar sin ayuda externa del los jugadores.

```
ggplot(df_long %>% filter(!(oferta == 0 & precio == 0)), aes(x = fecha, y = precio)) +
  geom_boxplot(fill = "skyblue") +
  scale_y_log10() +
  theme_minimal(base_size = 14) +
  labs(
    title = "Precio mensual de las skins (escala log)",
    x = "Mes",
    y = "Precio (CN¥)"
  ) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

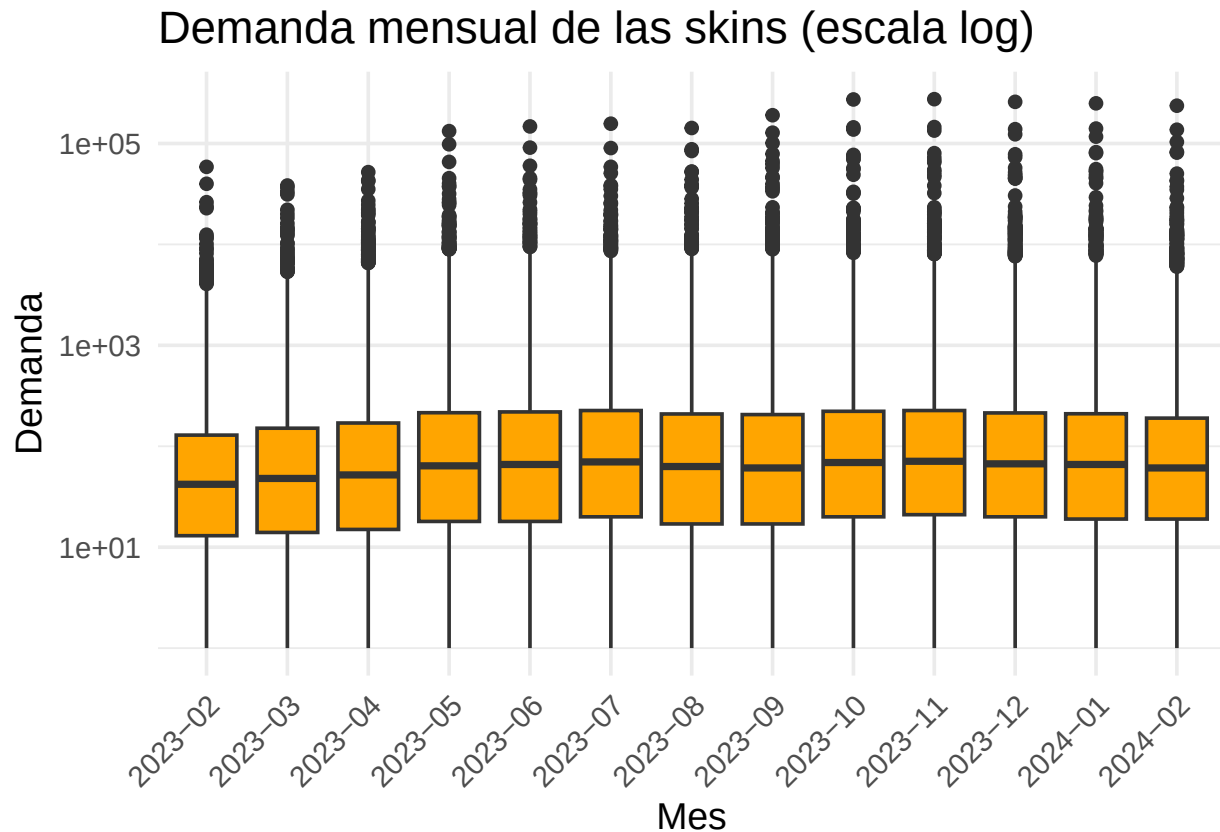


Podemos ver como los valores extremos se reducen (en distancia y cantidad) a medida que pasa el tiempo a su vez que crecen las cajas e intervalos a medida que pasa el tiempo, indicando un rango de valores más distribuidos.

```
ggplot(df_long %>% filter(!(oferta == 0 & precio == 0), fecha >= "2023-02")) , aes(x = fecha, y = oferta)
  geom_boxplot(fill = "orange") +
  scale_y_log10() +
  theme_minimal(base_size = 14) +
  labs(
    title = "Demanda mensual de las skins (escala log)",
    x = "Mes",
    y = "Demanda"
  ) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

```
## Warning in scale_y_log10(): log-10 transformation introduced infinite values.
```

```
## Warning: Removed 9975 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```



Para este caso vemos lo contrario, las cajas tienden a subir y la distribución a cerrarse. A su vez aumenta el numero de extremos y su distancia con el resto de valores, indicando una saturación de algunos items (exactamente lo que esperamos de las “bot farms” el mercado se infla de “drops” más comunes ya que para estos usuarios de IAs no les he de uso).

Normalidad

```
df_year_weapon <- df_long %>%
  mutate(
    fecha = ym(fecha),
    year = year(fecha)
  ) %>%
  group_by(year, nombre) %>%
  summarise(
    media_precio = mean(precio, na.rm = TRUE),
    media_demanda = mean(oferta, na.rm = TRUE),
    .groups = "drop"
  )

df_year_weapon %>%
  filter(media_precio > 0) %>% # para log10
  mutate(x = log10(media_precio)) %>%
  group_by(year) %>%
  summarise(
```

```

n = n(),
shapiro_p = if (n() <= 5000) shapiro.test(x)$p.value else shapiro.test(sample(x, 5000))$p.value,
ad_p      = nortest::ad.test(x)$p.value,
lillie_p  = nortest::lillie.test(x)$p.value,
.groups = "drop"
)

```

```

## # A tibble: 4 x 5
##   year      n shapiro_p      ad_p  lillie_p
##   <dbl> <int>      <dbl>      <dbl>      <dbl>
## 1  2021 13682  2.25e-17  8.12e-24  7.15e-14
## 2  2022 19490  8.87e-21  3.7 e-24  7.06e-155
## 3  2023 21919  6.01e-20  3.7 e-24  3.06e-154
## 4  2024 21953  5.22e-20  3.7 e-24  2.90e-147

```

```

df_year_weapon_dem <- df_long %>%
  mutate(fecha = ym(fecha)) %>%
  filter(fecha >= ym("2023-02")) %>%
  mutate(year = year(fecha)) %>%
  group_by(year, nombre) %>%
  summarise(media_demanda = mean(oferta, na.rm = TRUE), .groups = "drop")

df_year_weapon_dem %>%
  mutate(x = log1p(media_demanda)) %>% # admite ceros
  group_by(year) %>%
  summarise(
    n = n(),
    shapiro_p = if (n() <= 5000) shapiro.test(x)$p.value else shapiro.test(sample(x, 5000))$p.value,
    ad_p      = nortest::ad.test(x)$p.value,
    lillie_p  = nortest::lillie.test(x)$p.value,
    .groups = "drop"
  )

```

```

## # A tibble: 2 x 5
##   year      n shapiro_p      ad_p  lillie_p
##   <dbl> <int>      <dbl>      <dbl>      <dbl>
## 1  2023 21954  5.06e-14  3.7e-24  3.88e-21
## 2  2024 21954  2.37e-12  3.7e-24  4.91e-33

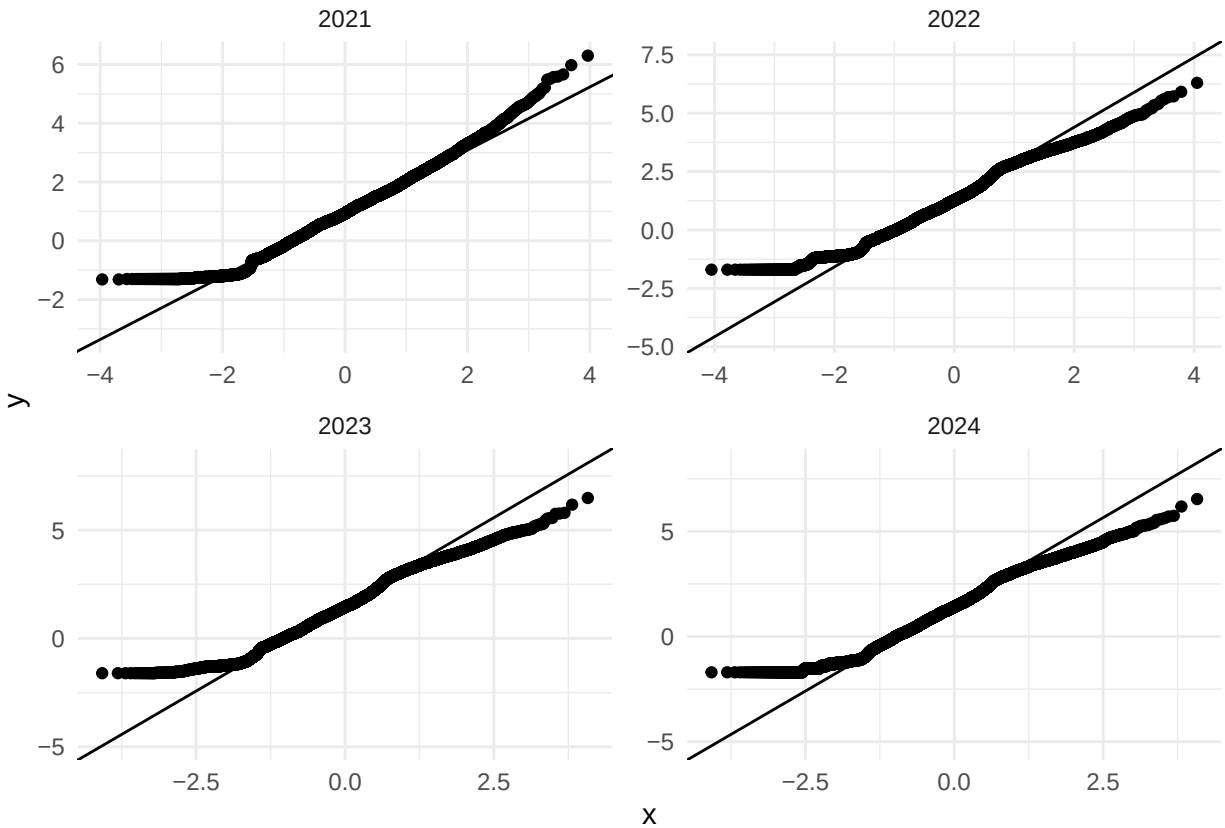
```

Al hacer los test vemos que los 2 primeros diagramas de la evolución del mercado pasan los tests para determinar la normalidad al tener un valor p muy pequeños (< 0.01).

```

df_year_weapon %>%
  filter(media_precio > 0) %>%
  mutate(x = log10(media_precio)) %>%
  ggplot(aes(sample = x)) +
  stat_qq() +
  stat_qq_line() +
  facet_wrap(~ year, scales = "free") +
  theme_minimal()

```

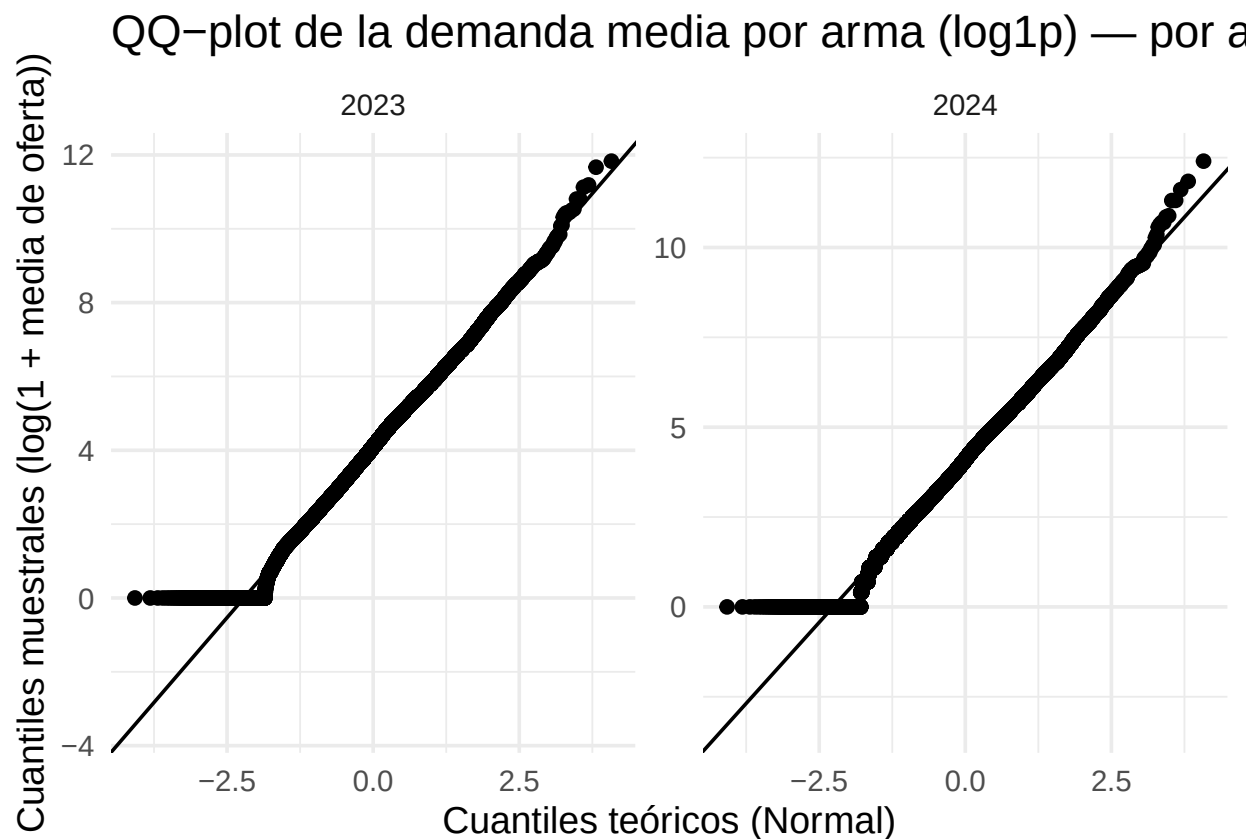


En este test de normalidad podemos ver que la gran mayoría de datos se pegan a la diagonal que representan a la normal, pero el extremo superior se separa lentamente por los eventos previamente mencionados y los inferiores pueden ser seguramente ignorados ya que sale de nuestro rango de precios (no existen los negativos).

```
df_year_weapon_dem %>%
  mutate(x = log1p(media_demanda)) %>%
  ggplot(aes(sample = x)) +
  stat_qq() +
  stat_qq_line() +
  facet_wrap(~ year, scales = "free") +
  theme_minimal(base_size = 14) +
  labs(
    title = "QQ-plot de la demanda media por arma (log1p) - por año",
    x = "Cuantiles teóricos (Normal)",
    y = "Cuantiles muestrales (log(1 + media de oferta))"
  )
```

```
## Warning: Removed 34 rows containing non-finite outside the scale range
## (`stat_qq()`).
```

```
## Warning: Removed 34 rows containing non-finite outside the scale range
## (`stat_qq_line()`).
```



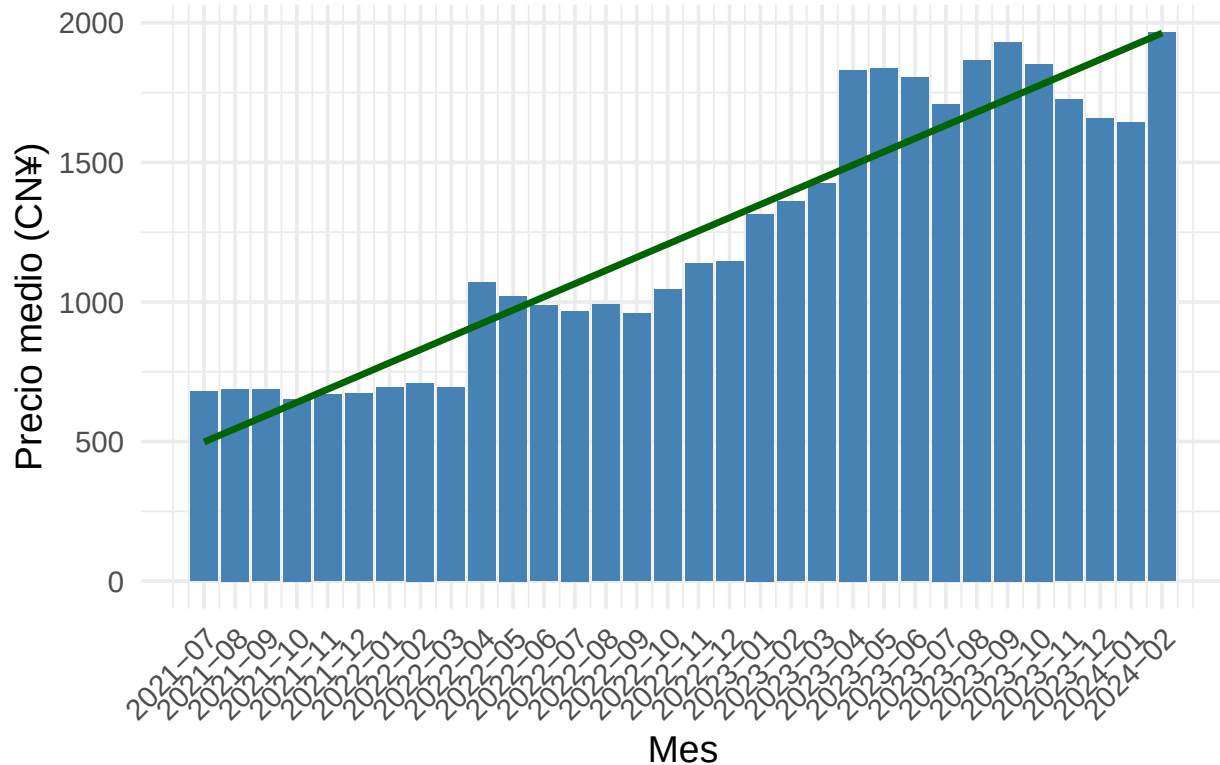
La demanda se pega más que los precios, con la misma desviación en ambos ejes. Nada fuera de lo esperable, lo único a destacar es como esta desviación empeora con el transcurso del tiempo.

Media y Varianza

```
ggplot(mercado_media, aes(x = as.numeric(factor(fecha)), y = media_precio)) +
  geom_col(aes(x = as.numeric(factor(fecha))), fill = "steelblue") +
  geom_smooth(method = "lm", se = FALSE, linewidth = 1.2, color = "darkgreen") +
  scale_x_continuous(
    breaks = seq_along(mercado_media$fecha),
    labels = mercado_media$fecha
  ) +
  theme_minimal(base_size = 14) +
  labs(
    title = "Evolución del precio medio mensual del mercado",
    x = "Mes",
    y = "Precio medio (CN¥)"
  ) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

Evolución del precio medio mensual del mercado



```
# Pendiente de la columna (crecimiento de media por mes)
mercado_media$mes_num <- seq_len(nrow(mercado_media))
modelo <- lm(media_precio ~ mes_num, data = mercado_media)

summary(modelo)
```

```
##
## Call:
## lm(formula = media_precio ~ mes_num, data = mercado_media)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -273.24 -118.37  -24.55   106.72   340.45
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   451.189     57.217    7.886 8.43e-09 ***
## mes_num       47.266      3.026   15.619 5.95e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 158.1 on 30 degrees of freedom
## Multiple R-squared:  0.8905, Adjusted R-squared:  0.8868
## F-statistic: 244 on 1 and 30 DF, p-value: 5.953e-16
```

```
# más detalles
```

```
round(coef(modelo)[["mes_num"]], 2)
```

```
## [1] 47.27
```

```
# El mercado suele subir 47.27(CN¥) por mes.
```

```
# ~50 centimos de euro a día de hoy.
```

Como podemos ver la regresión lineal se ajusta bien a los datos de media y nos da una evolución mensual de 47.27 yuanes.

```
mercado_media %>%  
  mutate(mes_num = row_number()) %>%  
  summarise(  
    grado_de_ajuste = summary(lm(media_precio ~ mes_num))$r.squared,  
    grado_de_ajuste_ajustado = summary(lm(media_precio ~ mes_num))$adj.r.squared,  
    raíz_del_error_cuadrático_medio = sqrt(mean(residuals(lm(media_precio ~ mes_num))^2))  
  )
```

```
## # A tibble: 1 x 3
```

```
##   grado_de_ajuste grado_de_ajuste_ajustado raíz_del_error_cuadrático_medio
```

```
##           <dbl>                <dbl>                <dbl>
```

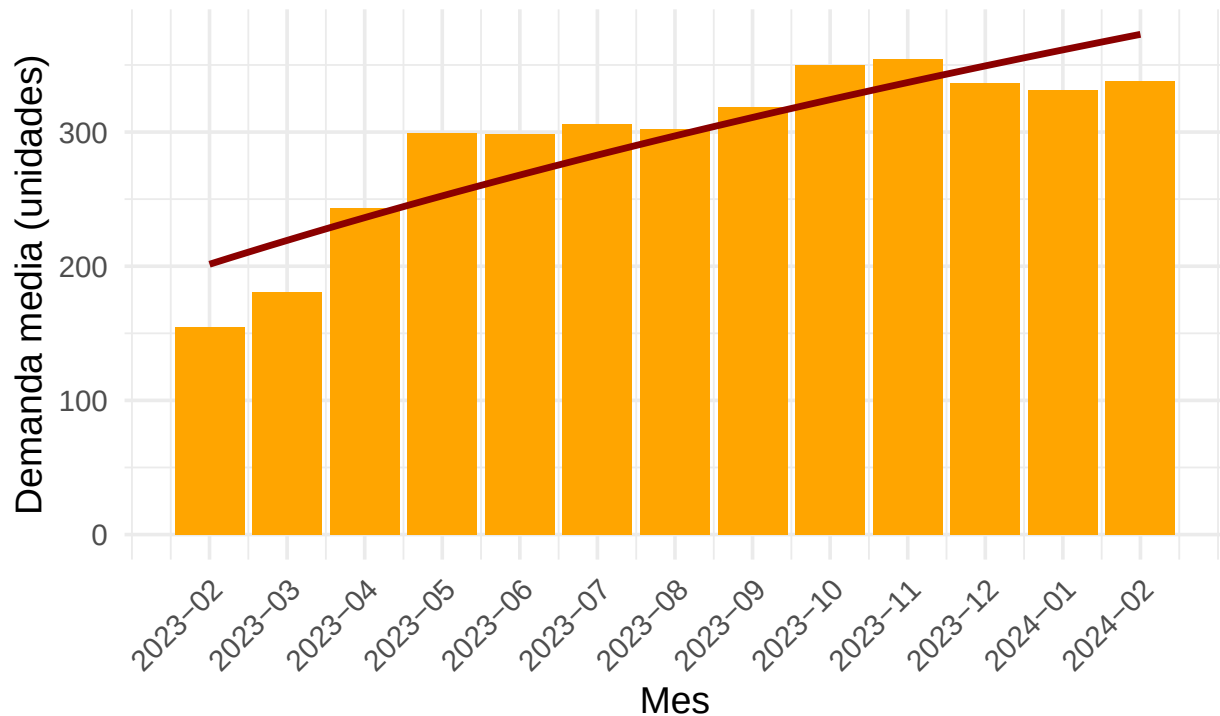
```
## 1           0.890                0.887                153.
```

```
# datos del ajuste cuadrático, se ajusta bien con un 0.89
```

```
ggplot(mercado_media_demanda, aes(x = mes_num, y = media_demanda)) +  
  geom_col(fill = "orange") +  
  stat_smooth(  
    method = "lm",  
    formula = y ~ log(x),  
    se = FALSE,  
    linewidth = 1.2,  
    color = "darkred"  
  ) +  
  scale_x_continuous(  
    breaks = mercado_media_demanda$mes_num,  
    labels = mercado_media_demanda$fecha  
  ) +  
  theme_minimal(base_size = 14) +  
  labs(  
    title = "Evolución de la demanda media mensual del mercado",  
    subtitle = "Ajuste logarítmico",  
    x = "Mes",  
    y = "Demanda media (unidades)"  
  ) +  
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```


Evolución de la demanda media mensual del mercado

Ajuste logarítmico



```
# Pendiente de la columna (crecimiento de media por mes)
modelo_log <- lm(media_demanda ~ log(mes_num), data = mercado_media_demanda)

summary(modelo_log)
```

```
##
## Call:
## lm(formula = media_demanda ~ log(mes_num), data = mercado_media_demanda)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -47.04 -30.13   6.80  23.29  46.99
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -890.82    193.25  -4.610 0.000753 ***
## log(mes_num)   364.61     59.45   6.134 7.38e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 31.3 on 11 degrees of freedom
## Multiple R-squared:  0.7738, Adjusted R-squared:  0.7532
## F-statistic: 37.62 on 1 and 11 DF,  p-value: 7.377e-05
```

```
# más detalles
```

```
round(coef(modelo_log)[["log(mes_num)"]], 2)
```

```
## [1] 364.61
```

```
# esto es para la fórmula de la pendiente:  $y^m = 0 + 1 \log(m)$ 
```

```
# y lo que calculamos es  $m = \text{mes\_num}$  y  $1 = 78.83$ .
```

```
# Tienes que hacer mates para sacarlo, pero claramente vemos que pasamos de ~50 a ~8.
```

```
mercado_media_demanda %>%
```

```
  summarise(
```

```
    grado_de_ajuste = summary(modelo_log)$r.squared,
```

```
    grado_de_ajuste_ajustado = summary(modelo_log)$adj.r.squared,
```

```
    raíz_del_error_cuadrático_medio = sqrt(mean(residuals(modelo_log)^2))
```

```
)
```

```
## # A tibble: 1 x 3
```

```
##   grado_de_ajuste grado_de_ajuste_ajustado raíz_del_error_cuadrático_medio
```

```
##           <dbl>                <dbl>                <dbl>
```

```
## 1           0.774                0.753                28.8
```

El ajuste cuadrático lineal aquí nos da un valor de similitud de 0.2, mientras que con el logarítmico lo podemos subir a un 0.75. Al ser logarítmico su pendiente es variable, pero podemos ver como en unos meses pasamos de una pendiente de 50 a uno de 8. Con los datos pasados por el summary se puede modelar para calcular cualquier punto intermedio o futuro si fuera necesario con la fórmula $y^m = 0 + 1 \log(m)$.

```
mercado_var <- df_long %>%
```

```
  filter(!(oferta == 0 & precio == 0)) %>%
```

```
  group_by(fecha) %>%
```

```
  summarise(
```

```
    var_precio = var(precio, na.rm = TRUE),
```

```
    var_demanda = var(oferta, na.rm = TRUE),
```

```
    n_obs = sum(!is.na(precio) & !is.na(oferta)),
```

```
    .groups = "drop"
```

```
) %>%
```

```
  arrange(fecha) %>%
```

```
  mutate(mes_num = row_number())
```

```
ggplot(mercado_var, aes(x = as.numeric(factor(fecha)), y = var_precio)) +
```

```
  geom_col(fill = "steelblue") +
```

```
  scale_x_continuous(
```

```
    breaks = seq_along(mercado_var$fecha),
```

```
    labels = mercado_var$fecha
```

```
) +
```

```
  theme_minimal(base_size = 14) +
```

```
  labs(
```

```
    title = "Evolución de la varianza mensual del precio del mercado",
```

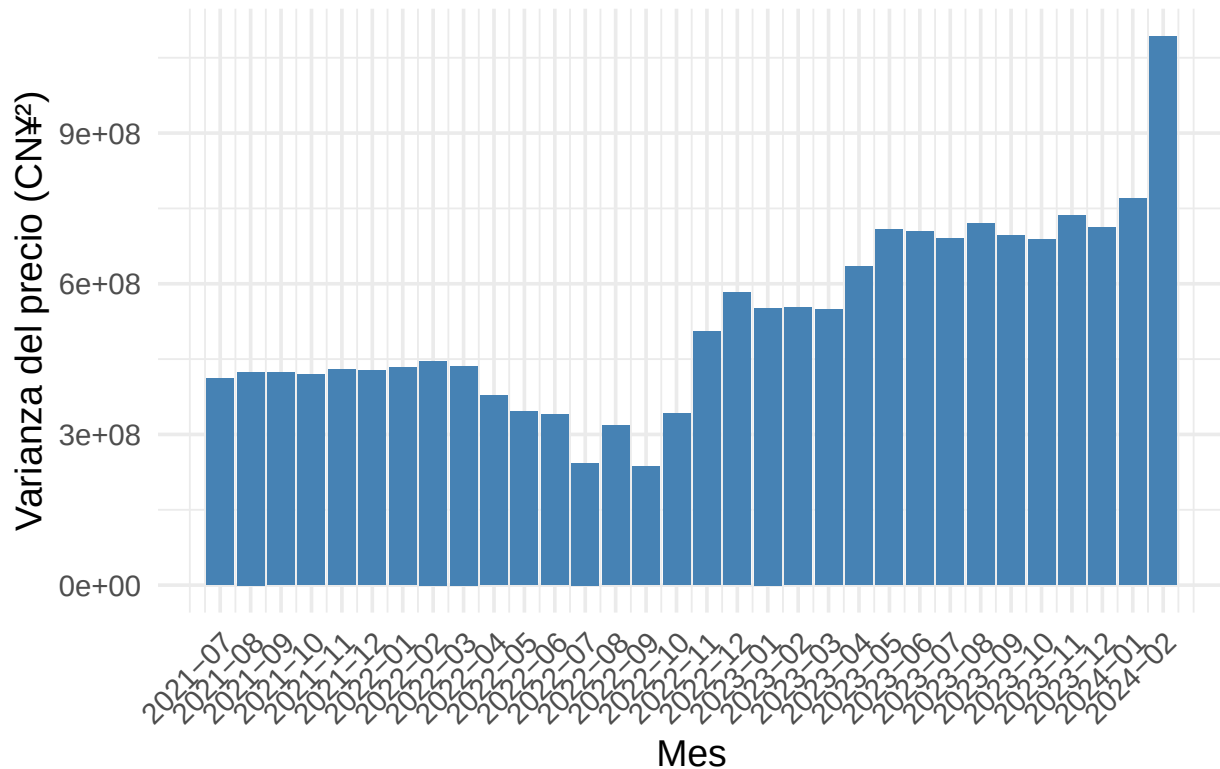
```
    x = "Mes",
```

```
    y = "Varianza del precio (CN¥²)"
```

```
) +
```

```
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

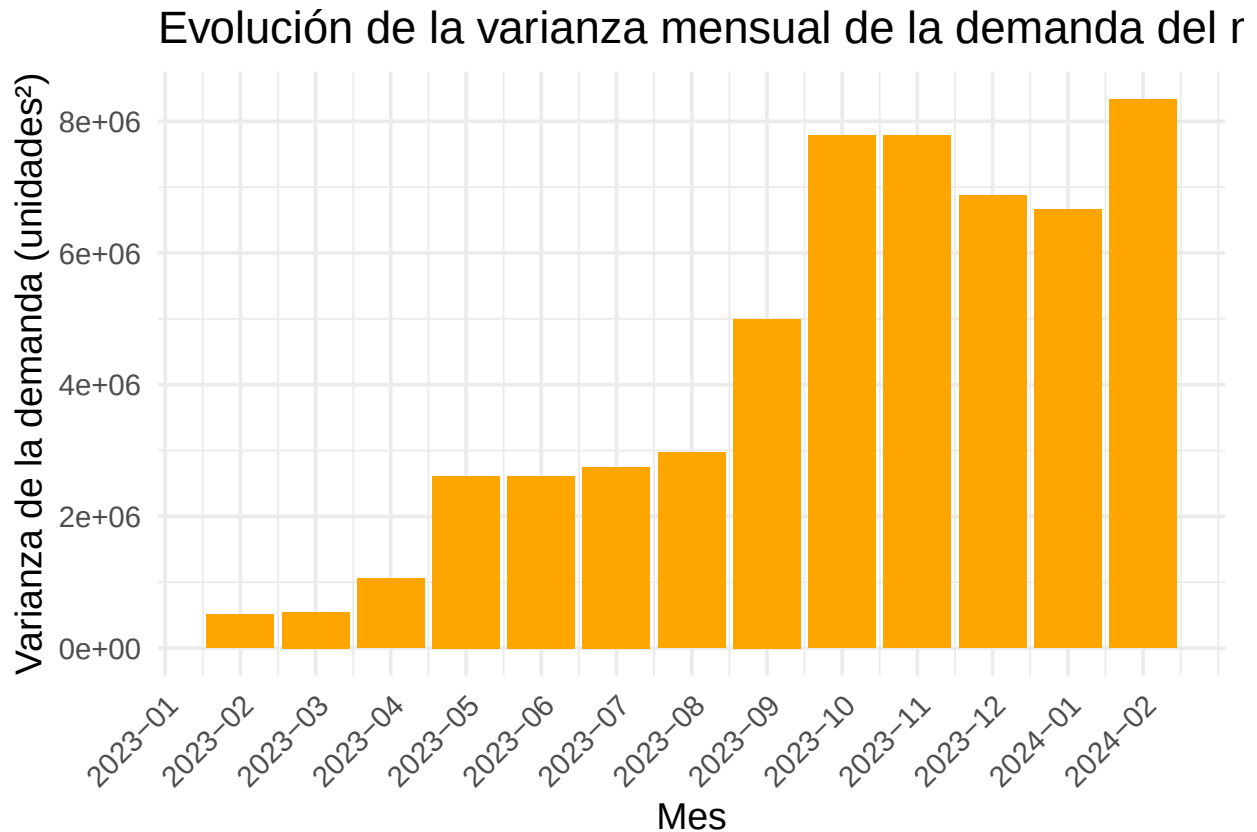
Evolución de la varianza mensual del precio del mercado



Podemos ver que crece a medida que el mercado lo hace, con una bajada en 2022 que a nuestro conocimiento se debía a un problema con una “sticker” que daba problemas y torneos que terminaron con gente haciendo trampas dejando al mercado más conservador en tiempos de tanta incertidumbre en como evolucionaría.

En 2024 en febrero se debe a un mes incompleto y terminó con una variación super alta, a su vez los bot farms se popularizaron empeorando la situación.

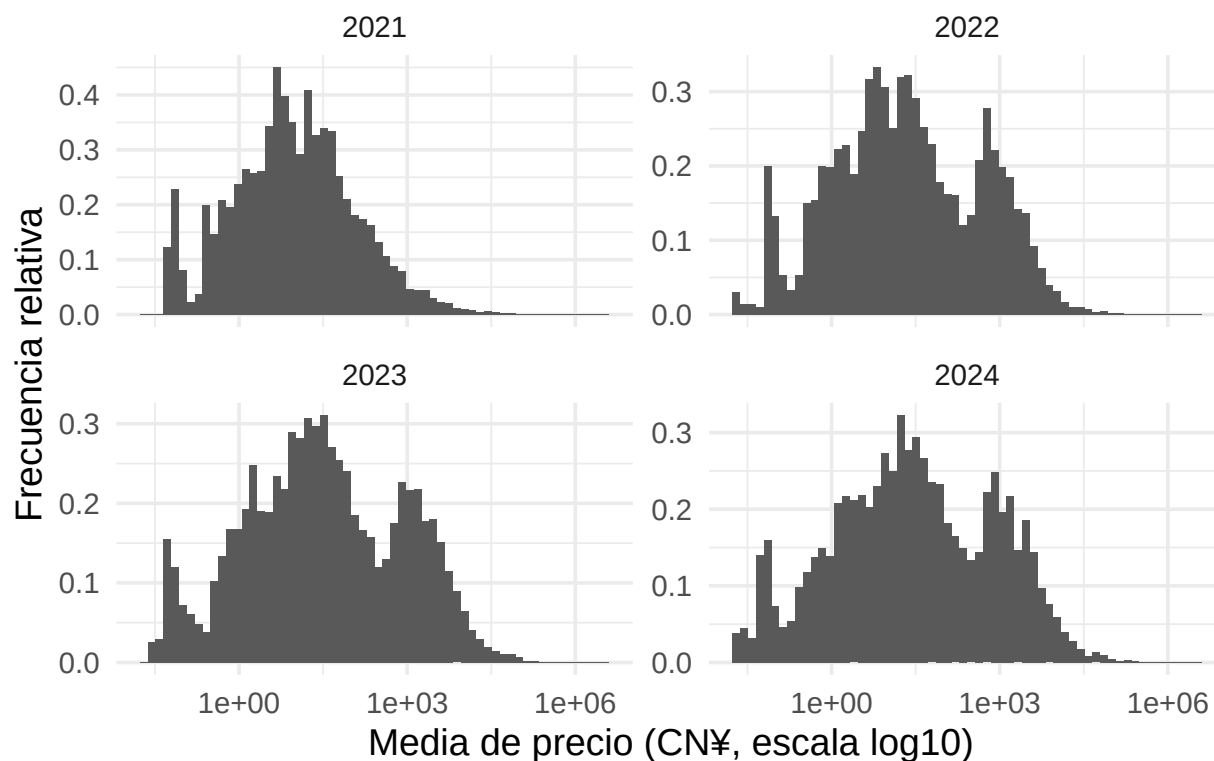
```
ggplot(mercado_var %>% filter(fecha >= "2023-02"), aes(x = mes_num, y = var_demanda)) +
  geom_col(fill = "orange") +
  scale_x_continuous(
    breaks = mercado_var$mes_num,
    labels = mercado_var$fecha
  ) +
  theme_minimal(base_size = 14) +
  labs(
    title = "Evolución de la varianza mensual de la demanda del mercado",
    x = "Mes",
    y = "Varianza de la demanda (unidades²)"
  ) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



La varianza explotó sobre este periodo ya que salieron armas que son de las más populares hasta el día de hoy haciendo más extremos y mucha especulación del valor real de estas nuevas y llamativas armas.

```
ggplot(df_year_weapon %>% filter(media_precio > 0),
  aes(x = media_precio)) +
  geom_histogram(aes(y = after_stat(density)), bins = 60) +
  scale_x_log10() +
  facet_wrap(~ year, scales = "free_y") +
  theme_minimal(base_size = 14) +
  labs(
    title = "Distribución de medias de valor por arma por año",
    x = "Media de precio (CN¥, escala log10)",
    y = "Frecuencia relativa"
  )
```

Distribución de medias de valor por arma por año



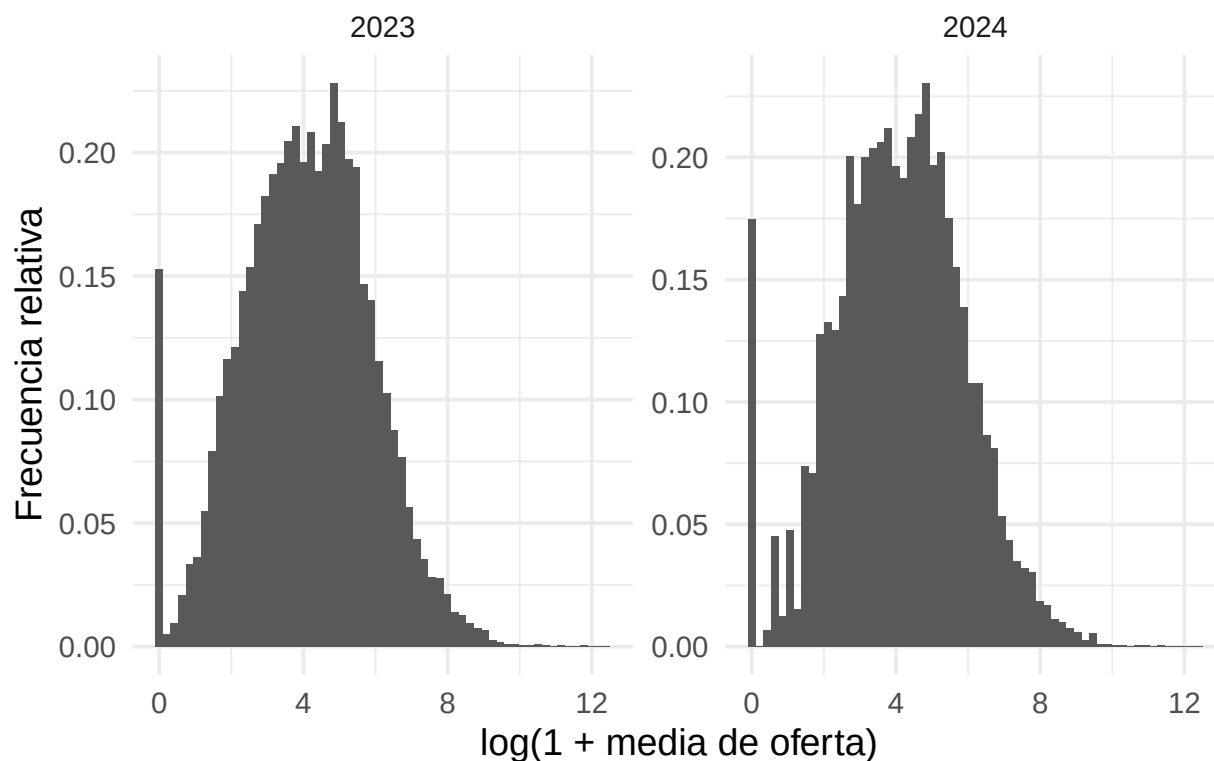
Podemos ver como la distribución de precios se aleja de una normal a una distribución de 3 montañas, barato, normal y caro y evidentemente la cola de las muy caras que lentamente aumenta y crece.

A su vez vemos que hay más items intermedios conectando estas 3 montañas una clara demostración de como han añadido más items y algunos han caído en estas zonas menos comunes uniendo más la distribución. Quizás con más items que la gente no demanda tanto podríamos esperar a ver menos picos y un mercado más normalizado. Un claro ejemplo como un mercado con muchas opciones tiende a normalizarse sobre uno con pocas.

```
ggplot(df_year_weapon_dem, aes(x = log1p(media_demanda))) +
  geom_histogram(aes(y = after_stat(density)), bins = 60) +
  facet_wrap(~ year, scales = "free_y") +
  theme_minimal(base_size = 14) +
  labs(
    title = "Distribución de medias de demanda por arma por año",
    x = "log(1 + media de oferta)",
    y = "Frecuencia relativa"
  )
```

```
## Warning: Removed 34 rows containing non-finite outside the scale range
## (`stat_bin()`).
```

Distribución de medias de demanda por arma por año



En la distribución podemos ver una distribución normal pero con 15%-20% de los items no tienen demanda al ser nuevos o raramente pedidos. Esto confronta la idea que más elección ayuda a normalizar el mercado, ya que normaliza los precios pero a su vez causa un problema de oferta ya que son menos probables de aparecer sobre otras y por lo tanto su venta es escasa por su baja demanda y precio.

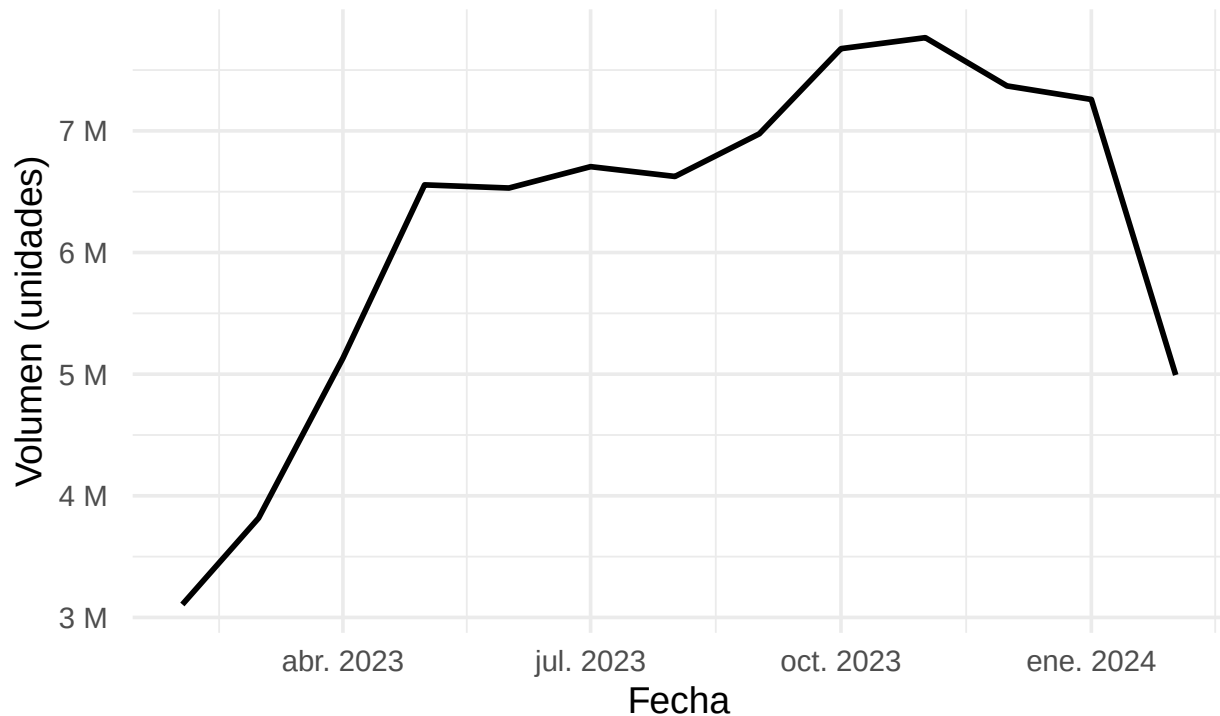
Métricas del mercado

Estás son métricas del mercado usados principalmente en economía, como no estamos formados en estos ámbitos simplemente comentaremos las conclusiones que podemos derivar de estos diagramas.

```
ggplot(metrics_plot, aes(x = fecha, y = volumen_unidades)) +  
  geom_line(linewidth = 1) +  
  scale_y_continuous(labels = fmt_si) +  
  theme_minimal(base_size = 14) +  
  labs(  
    title = "Actividad del mercado",  
    subtitle = "Volumen mensual (unidades vendidas)",  
    x = "Fecha", y = "Volumen (unidades)"  
  )
```

Actividad del mercado

Volumen mensual (unidades vendidas)



Eje X: tiempo (meses); Eje Y: unidades vendidas (millones)

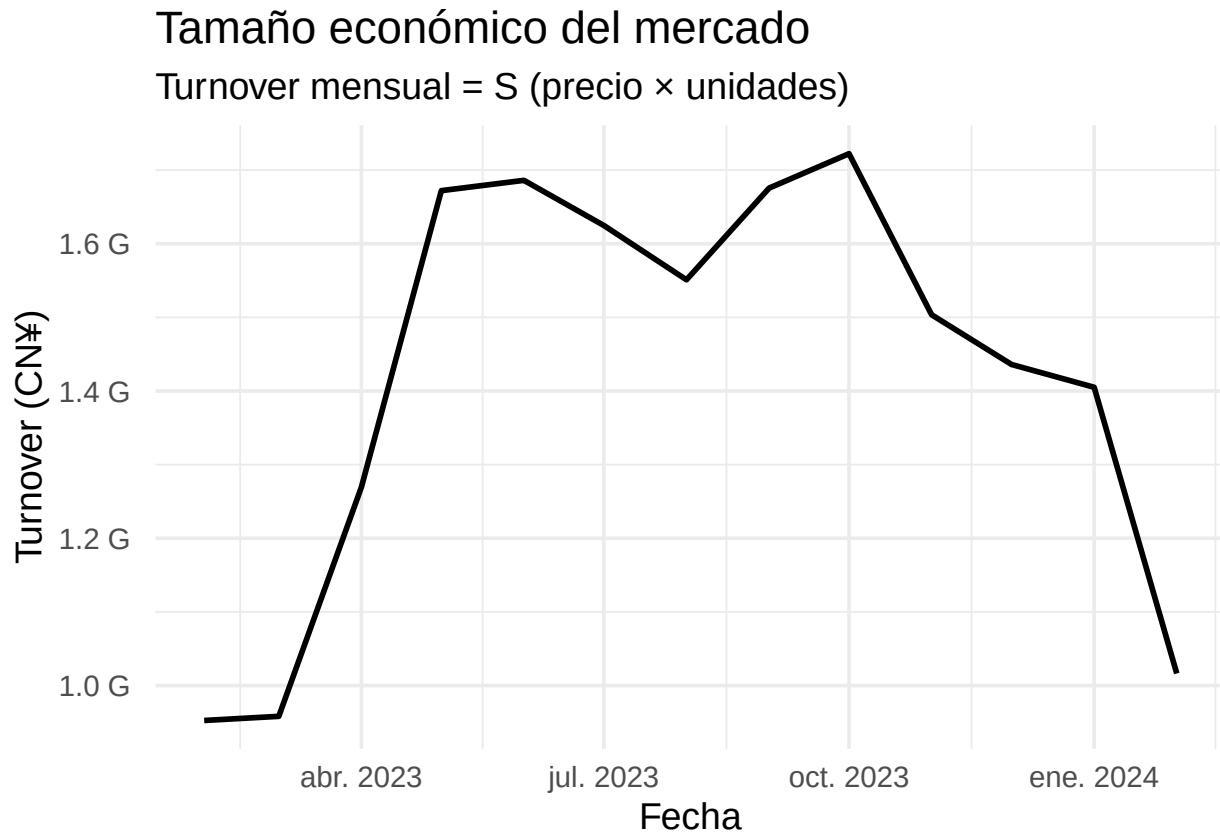
¿Qué mide? El número total de unidades vendidas cada mes (sin tener en cuenta el precio) -> Es una medida pura de actividad.

Se observa:

- El mercado crece rápidamente al inicio (fase de expansión).
- Luego entra en una meseta de actividad alta.
- caída brusca en el último mes por lo mencionado anteriormente.

Pero actividad ≠ valor económico, de ahí el siguiente.

```
ggplot(metrics_plot, aes(x = fecha, y = turnover)) +  
  geom_line(linewidth = 1) +  
  scale_y_continuous(labels = fmt_si) +  
  theme_minimal(base_size = 14) +  
  labs(  
    title = "Tamaño económico del mercado",  
    subtitle = "Turnover mensual =  $\Sigma$  (precio  $\times$  unidades)",  
    x = "Fecha", y = "Turnover (CN¥)"  
  )
```



¿Qué mide? El valor monetario total del mercado cada mes; mide el “tamaño económico” real.

Eje X: tiempo (meses); Eje Y en CNY (miles de millones)

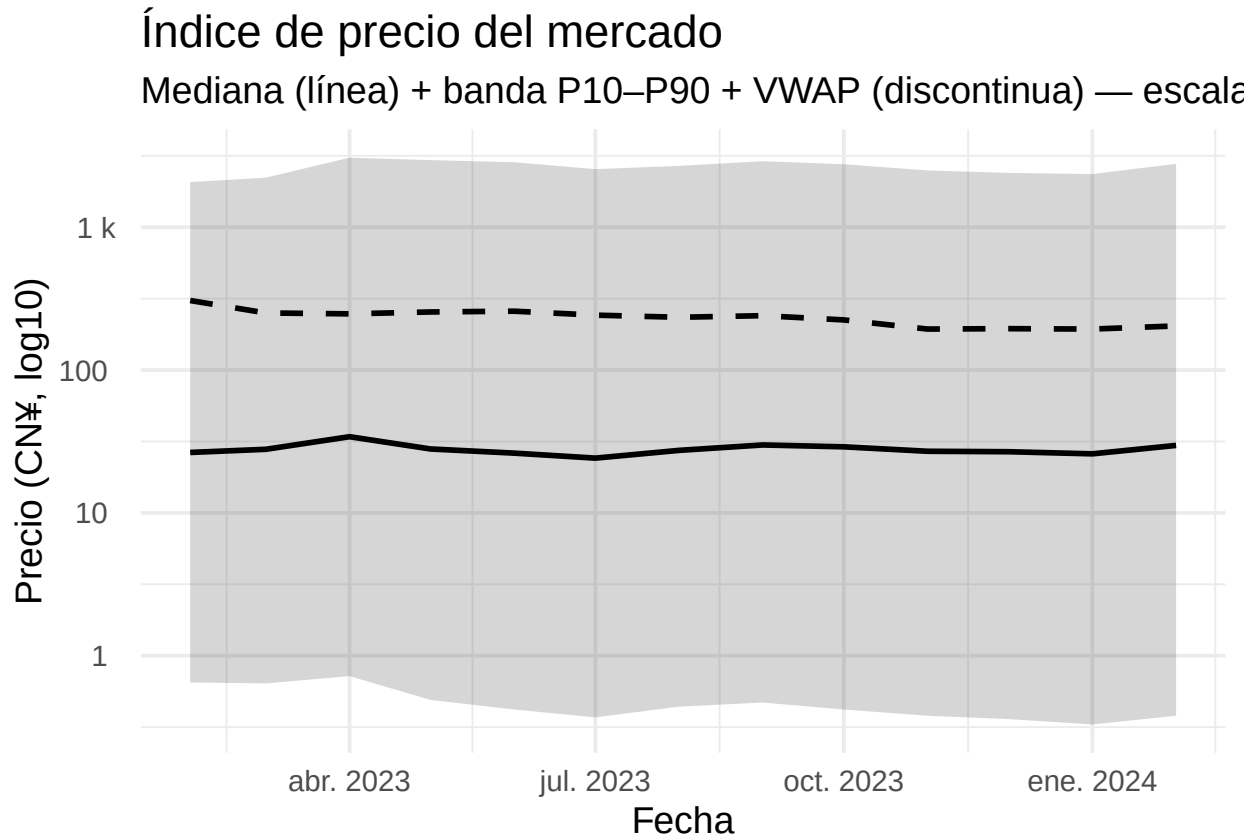
Se observa:

Que aunque el volumen se mantiene alto, el turnover cae más, lo que sugiere: precios medios más bajos o desplazamiento hacia ítems baratos.

El mercado sigue activo, pero mueve menos dinero. Esto es típico de:

- maduración del mercado
- menor especulación
- mayor rotación de ítems comunes

```
ggplot(metrics_plot, aes(x = fecha)) +
  geom_ribbon(aes(ymin = p10_precio, ymax = p90_precio), alpha = 0.20) +
  geom_line(aes(y = mediana_precio), linewidth = 1) +
  geom_line(aes(y = vwap), linewidth = 1, linetype = "dashed") +
  scale_y_log10(labels = fmt_si) +
  theme_minimal(base_size = 14) +
  labs(
    title = "Índice de precio del mercado",
    subtitle = "Mediana (línea) + banda P10-P90 + VWAP (discontinua) - escala log",
    x = "Fecha", y = "Precio (CN¥, log10)"
  )
```

Qué mide cada elemento:

Línea continua → mediana del precio

Banda gris (P10–P90) → dispersión de precios

Línea discontinua → VWAP (precio medio ponderado por volumen)

Escala logarítmica → necesaria por la enorme heterogeneidad de precios

Se observa:

La mediana es estable → el “ítem típico” no cambia mucho de precio

La VWAP está muy por encima → pocos ítems caros concentran valor

La banda P10–P90 es amplia → mercado muy desigual

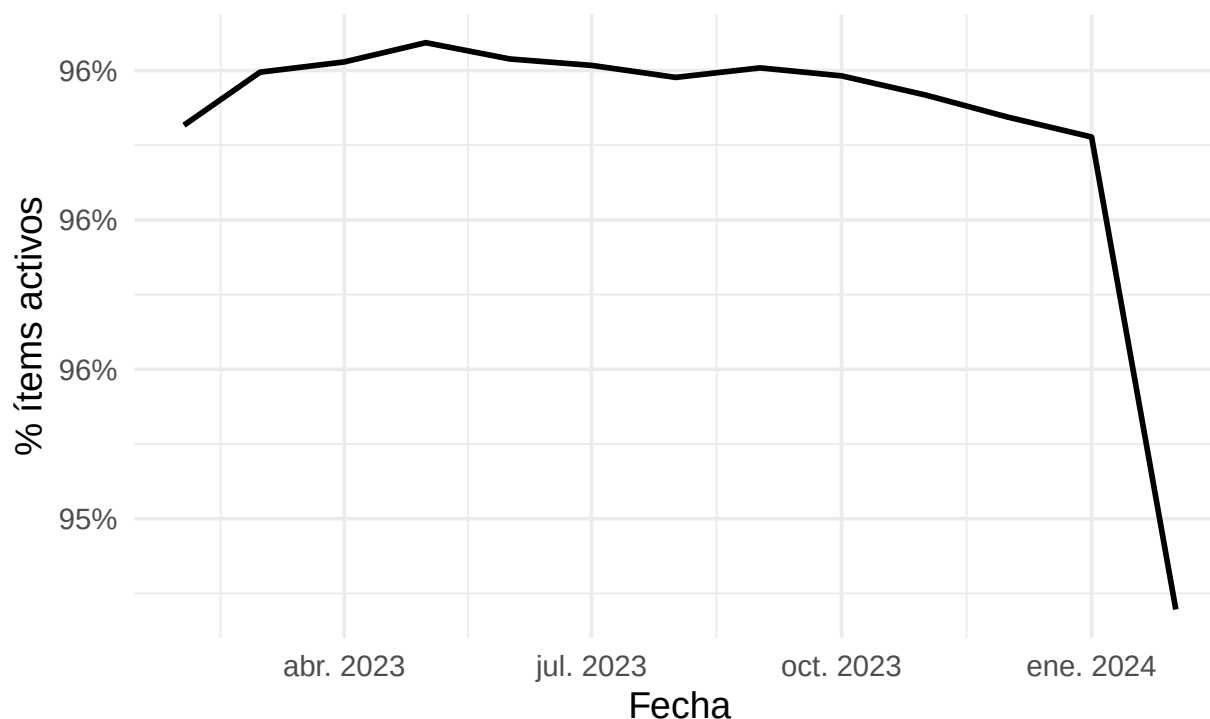
Es decir, el precio “normal” es estable y el valor total del mercado depende de pocos ítems caros; esto explica por qué: el turnover puede caer aunque el volumen no lo haga tanto.

“Mercado estable en el centro, muy asimétrico en la cola”

```
ggplot(metrics_plot, aes(x = fecha, y = prop_activos)) +
  geom_line(linewidth = 1) +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  theme_minimal(base_size = 14) +
  labs(
    title = "Liquidez global (breadth)",
    subtitle = "% de ítems con ventas sobre ítems con datos",
    x = "Fecha", y = "% ítems activos"
  )
```

Liquidez global (breadth)

% de ítems con ventas sobre ítems con datos



¿Qué mide? El porcentaje de ítems se vende al menos una vez en el mes.

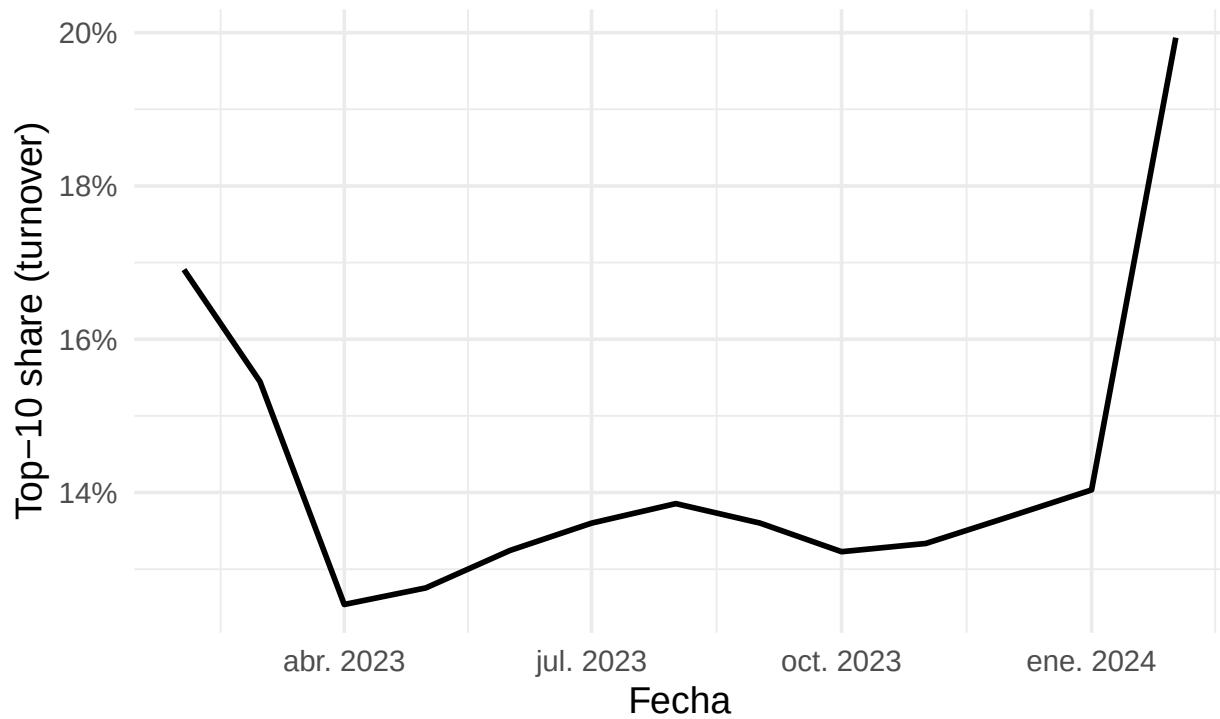
Se observa:

- Una pequeña caída de 1% pero con una inclinación preocupante implicando que podría seguir así en el futuro, esto se debió al gran flujo de nuevos ítems y la gente siendo conservador en no querer ponerlos a la venta por debajo de un mejor valor futuro.

```
ggplot(metrics_plot, aes(x = fecha, y = top10_share_val)) +  
  geom_line(linewidth = 1) +  
  scale_y_continuous(labels = percent_format(accuracy = 1)) +  
  theme_minimal(base_size = 14) +  
  labs(  
    title = "Concentración del mercado",  
    subtitle = "Share del Top-10 por turnover",  
    x = "Fecha", y = "Top-10 share (turnover)"  
  )
```

Concentración del mercado

Share del Top-10 por turnover



¿Qué mide? El porcentaje del valor total lo generan los 10 items más vendidos; es una medida clásica de concentración.

Se observa:

Que el mercado pasa de ser dominado por pocos items a ser más diversificado

El salto final indica:

- huida del valor hacia items “seguros”
- o caída general donde solo sobreviven los top

Esto claramente es una señal de estrés o una transición de mercado.

```
ggplot(metrics_plot, aes(x = fecha, y = cor_log_precio_dem)) +  
  geom_hline(yintercept = 0, linewidth = 0.6) +  
  geom_line(linewidth = 1) +  
  theme_minimal(base_size = 14) +  
  labs(  
    title = "Relación precio-demanda (por mes)",  
    subtitle = "Correlación cross-section: cor(log(precio), log(1 + oferta))",  
    x = "Fecha", y = "Correlación"  
  )
```



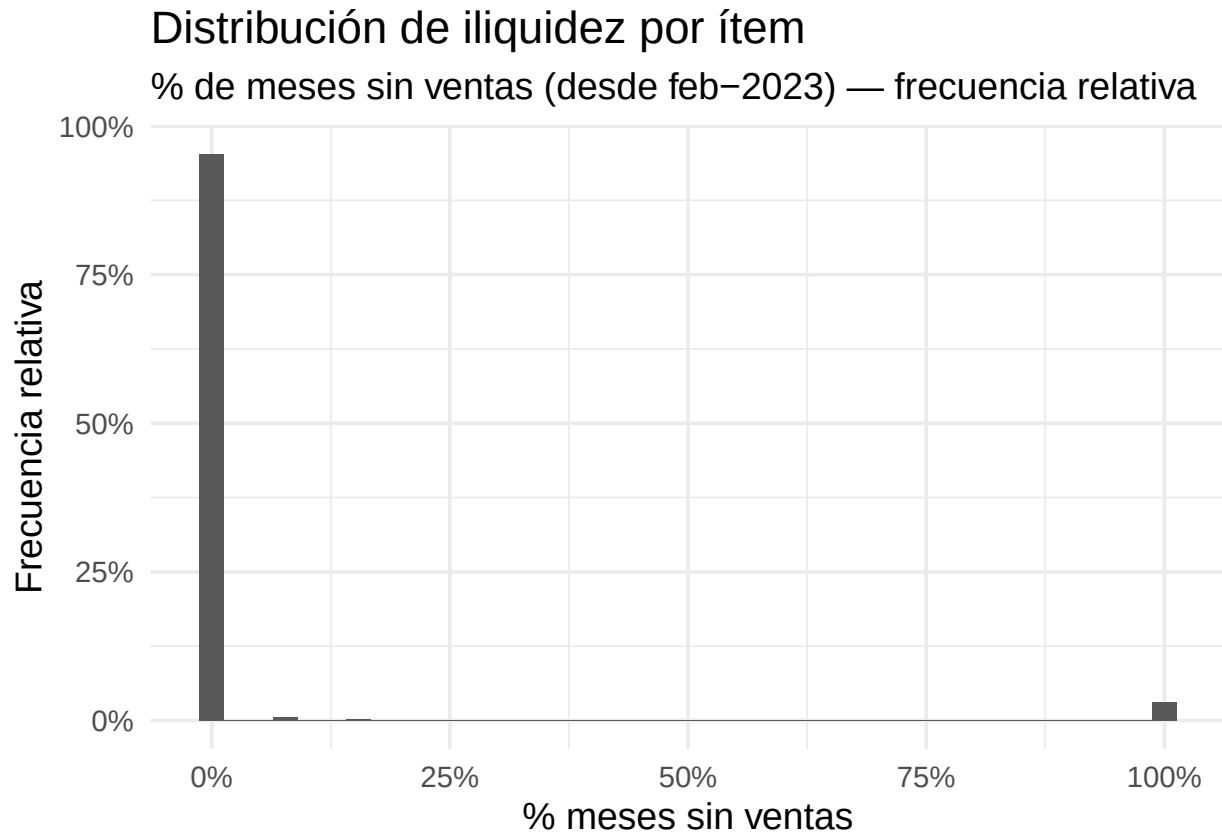
¿Qué mide? Relación cross-section mensual entre: precio y liquidez (oferta/demanda).

Se observa:

- Correlación negativa y estable
- Ítems más líquidos → precios más bajos
- Refuerza la lógica de mercado competitivo

Exactamente lo esperable de un mercado de este tipo.

```
ggplot(illiq_plot, aes(x = prop_meses_sin_ventas)) +
  geom_histogram(aes(y = after_stat(count / sum(count))), bins = 40) +
  scale_x_continuous(labels = percent_format(accuracy = 1)) +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  theme_minimal(base_size = 14) +
  labs(
    title = "Distribución de iliquidez por ítem",
    subtitle = "% de meses sin ventas (desde feb-2023) - frecuencia relativa",
    x = "% meses sin ventas",
    y = "Frecuencia relativa"
  )
```



¿Qué mide? En qué proporción del período no se vendió.

Se observa:

Pico enorme en 0% → la mayoría siempre se vende

Pequeño pico en 100% → ítems muertos

Casi nada en medio → mercado “bimodal”

Es decir hay muchos ítems activos y una cola de ítems estructuralmente ilíquidos. No es un mercado “normal” como los que encuentras en la bolsa, es todo o nada.

Loot boxes

Para tratar este apartado de nuestro trabajo, se estudiará las probabilidades y rentabilidad de una caja de Counter-Strike en particular, el Fracture Case.

A la hora de abrir una caja, se producen 3 sorteos:

- Un sorteo para decidir qué ítem de la caja es elegido.
- Un sorteo para decidir si tiene StatTrak el ítem elegido o no.
- Un sorteo para decidir qué desgaste tiene el ítem elegido.

Una vez realizado los 3 sorteos, el ítem es entregado al usuario.

Primer sorteo Cada ítem en toda caja de Counter-Strike se clasifica en 1 de 5 categorías, de más a menos común: Mil-Spec, Restricted, Classified, Covert y Special Item. Cada categoría tiene 5 veces más probabilidades de ser elegido que el anterior. La tabla de probabilidades se vería así:

```
probs <- data.frame(category=c("Mil-Spec", "Restricted", "Classified", "Covert", "Special Item"), prob=
probs
```

```
##      category    prob
## 1   Mil-Spec 0.79923
## 2  Restricted 0.15985
## 3  Classified 0.03197
## 4      Covert 0.00639
## 5 Special Item 0.00256
```

En una caja, la probabilidad individual de cada item de ser elegido se calcula como la división de la probabilidad de que sea elegida la categoría a la que pertenece entre el número de items que pertenecen a dicha categoría en la caja. En el caso del Fracture Case:

```
fracture_weapons <- c("Negev | Ultralight",
  "P2000 | Gnarled",
  "SG 553 | Ol' Rusty",
  "SSG 08 | Mainframe 001",
  "P250 | Cassette",
  "P90 | Freight",
  "PP-Bizon | Runic",
  "MAG-7 | Monster Call",
  "Tec-9 | Brother",
  "MAC-10 | Allure",
  "Galil AR | Connexion",
  "MP5-SD | Kitbash",
  "M4A4 | Tooth Fairy",
  "Glock-18 | Vogue",
  "XM1014 | Entombed",
  "Desert Eagle | Printstream",
  "AK-47 | Legion of Anubis",
  "Special Item")

rarity <- c(7, 5, 3, 2, 1) # Cuantos items de la caja pertenecen a cada categoría (los 7 primeros a Mil

fracture_case <- data.frame(weapons=fracture_weapons, probs=rep(probs$prob/rarity, rarity))

fracture_case
```

```
##      weapons    probs
## 1   Negev | Ultralight 0.11417571
## 2    P2000 | Gnarled 0.11417571
## 3   SG 553 | Ol' Rusty 0.11417571
## 4   SSG 08 | Mainframe 001 0.11417571
## 5    P250 | Cassette 0.11417571
## 6     P90 | Freight 0.11417571
## 7   PP-Bizon | Runic 0.11417571
## 8   MAG-7 | Monster Call 0.03197000
## 9    Tec-9 | Brother 0.03197000
## 10   MAC-10 | Allure 0.03197000
## 11   Galil AR | Connexion 0.03197000
```

```
## 12          MP5-SD | Kitbash 0.03197000
## 13          M4A4 | Tooth Fairy 0.01065667
## 14          Glock-18 | Vogue 0.01065667
## 15          XM1014 | Entombed 0.01065667
## 16 Desert Eagle | Printstream 0.00319500
## 17 AK-47 | Legion of Anubis 0.00319500
## 18          Special Item 0.00256000
```

Segundo sorteo Como mencionado anteriormente, el segundo sorteo se realiza una vez elegido el arma para decidir si este tiene StatTrak o no. StatTrak no es nada más que un contador que se le añade a una skin que lo hace más valiosa que una sin StatTrak.

Para decidir si un arma tiene StatTrak sencillamente se comprueba si un número aleatorio generado entre 0 y 1 es inferior a 0.10, es decir, existe un 10% de probabilidad de que el item tenga StatTrak. En R la implementación se haría así con la distribución uniforme:

```
runif(1) < 0.10
```

Tercer sorteo El último sorteo es para determinar el desgaste del arma elegido. Cuanto menos desgastado, más valioso es el arma en precio. El desgaste de un arma se calcula con la siguiente formula:

$$Wear = Min + (Random \times (Max - Min))$$

Donde Min y Max es el desgaste mínimo y máximo que puede tener el arma elegido (son valores programados internamente en el juego) y Random es un número aleatorio entre 0 y 1. En R la ecuación se implementaría algo así:

```
fracture_case$min_wear = c(0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00,
                           0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00) # los desgastes minimos de l
fracture_case$max_wear = c(0.78, 1.00, 1.00, 1.00, 0.60, 1.00, 1.00, 1.00, 1.00, 1.00,
                           0.80, 0.80, 0.73, 0.75, 0.50, 0.80, 0.70, 1.00) # los desgastes maximos de l
```

```
fracture_case$min_wear + (runif(1) * (fracture_case$max_wear - fracture_case$min_wear))
```

En función del valor de desgaste calculado, el desgaste se clasifica en 1 de 5 categorías de menor a mayor desgaste: Factory New, Minimal Wear, Field-Tested, Well-Worn y Battle-Scarred. Para que el desgaste se pueda clasificar, este tiene que caer en uno de los siguientes rangos:

```
wear <- list(
  names = c("Factory New", "Minimal Wear", "Field-Tested", "Well-Worn", "Battle-Scarred"),
  breaks = c(0.00, 0.07, 0.15, 0.38, 0.45, 1.00)
)

data.frame(wear$names, lower = c(0.00, 0.07, 0.15, 0.38, 0.45),
           upper = c(0.07, 0.15, 0.38, 0.45, 1.00))
```

```
##      wear.names lower upper
## 1   Factory New  0.00  0.07
## 2  Minimal Wear  0.07  0.15
## 3  Field-Tested  0.15  0.38
## 4    Well-Worn  0.38  0.45
## 5 Battle-Scarred 0.45  1.00
```

Rentabilidad Ahora procederemos a simular la rentabilidad de abrir una cantidad determinada de cajas Fracture. Para ello crearemos un pequeño simulador en el que sacamos `n_items` de la caja en una fecha de precios determinada llamada `date`:

```
gamble <- function(n_items, date) {
  samp <- data.frame(weapons=sample(fracture_case$weapons, n_items, replace=T, fracture_case$probs)) #

  samp$stat = runif(n_items) < 0.10 # Tiene StatTrak? (10% probabilidad)

  indexes <- match(samp$weapons, fracture_case$weapons)
  samp$wear <- cut(fracture_case$min_wear[indexes] + (runif(nrow(samp)) * (fracture_case$max_wear[indexes] - fracture_case$min_wear[indexes])),
                  wear$breaks,
                  wear$names) # Calculamos el desgaste del arma y clasificamos ese desgaste

  df_date <- df_long_filtered %>% filter(fecha == date) # Cogemos los precios de un mes en particular

  samp$weapons <- ifelse(
    samp$weapons == "Special Item",

    sample((df_date %>% filter(grepl("Knife", nombre))) %>% pull(nombre), n_items), # Cogemos un cuchillo

    paste0(ifelse(samp$stat, "StatTrak ", ""), samp$weapons, " (", samp$wear, ")") # Construimos la etiqueta

  )

  samp$price <- (df_date %>% pull(precio))[match(samp$weapons, df_date %>% pull(nombre))] # Cogemos el precio

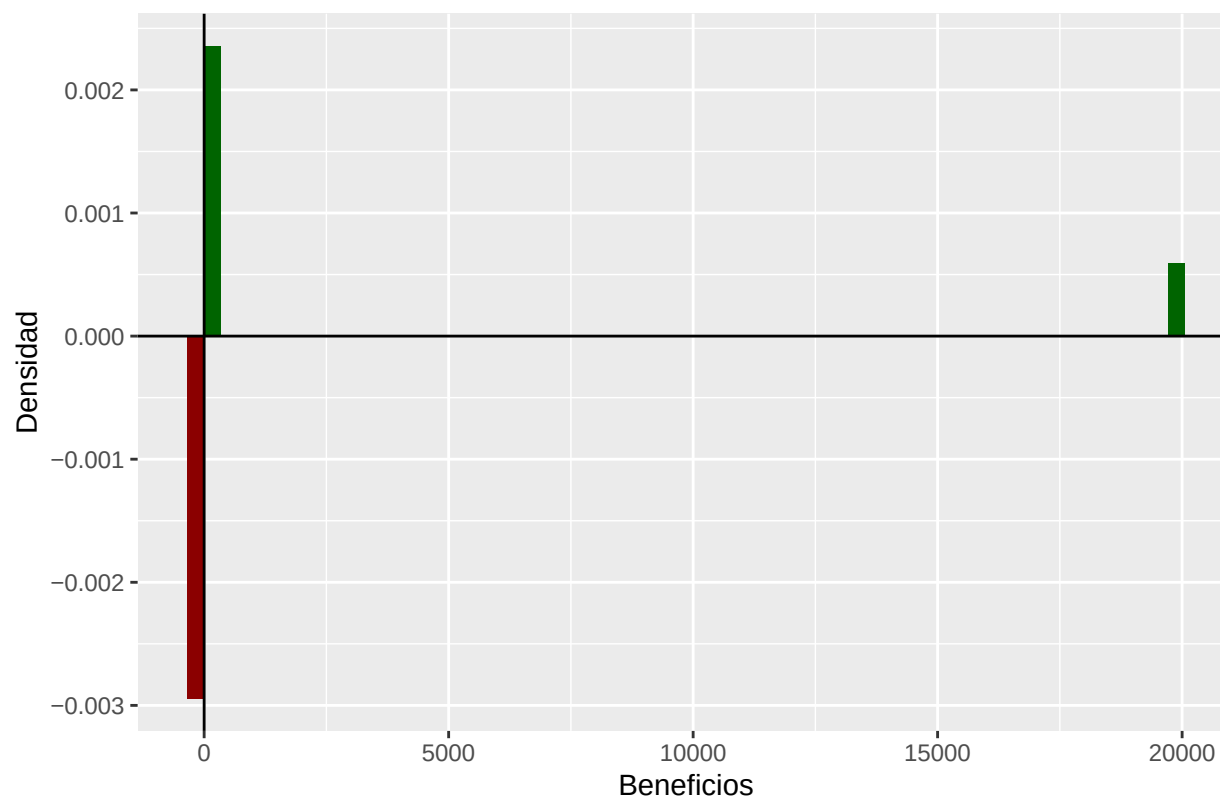
  return(samp) # Retornamos el dataframe conteniendo todos los items que hemos sacado de la caja
}
```

Ahora graficaremos la rentabilidad de abrir 10 cajas en 100 simulaciones distintas usando los valores de precio de por ejemplo marzo del 2023. Tenemos que tener en cuenta el precio de una llave de la caja Fracture cuesta 2.50\$ o 17.60 yuanes. Debemos restarlo a los precios de los items conseguidos en cada simulación para conseguir el beneficio real de abrir las cajas.

```
beneficios <- replicate(100, sum(gamble(10, "2023-03")$price) - 17.60 * 10) # 100 simulaciones en las que abrimos 10 cajas

ggplot() + geom_histogram(aes(x=beneficios[beneficios > 0], y=..density..), fill="darkgreen", bins = 60) +
  geom_histogram(aes(x=beneficios[beneficios < 0], y=-..density..), fill="darkred", bins = 60) +
  labs(title="Beneficios de abrir 10 cajas para 100 simulaciones", x="Beneficios", y="Densidad") +
  geom_hline(yintercept = 0) +
  geom_vline(xintercept = 0)
```


Beneficios de abrir 10 cajas para 100 simulaciones



Se puede observar que existen grandes pérdidas puesto que la densidad de beneficios se concentra debajo del 0. Además las ganancias que sí se consiguen suelen ser con un beneficio relativamente pequeño, aunque existe la posibilidad de conseguir beneficios muy grandes al haber sacado un item special o una skin con mucha rareza. Podemos ver numericamente los beneficios de todas las simulaciones y ver que en la mayoría de simulaciones se pierde dinero. La suma de los beneficios puede dar positivo en el caso de que uno tenga mucha suerte, pero si se corren más simulaciones los beneficios tenderán a ser negativos.

```
data.frame(simulación=1:100,beneficios)
```

##	simulación	beneficios
## 1	1	-135.69
## 2	2	-166.63
## 3	3	-127.14
## 4	4	-165.75
## 5	5	-140.07
## 6	6	-131.88
## 7	7	-146.38
## 8	8	-167.62
## 9	9	-166.27
## 10	10	-136.60
## 11	11	-166.81
## 12	12	-165.82
## 13	13	-164.70
## 14	14	-168.29
## 15	15	-167.64
## 16	16	-129.61

## 17	17	-145.44
## 18	18	-167.44
## 19	19	-170.35
## 20	20	-122.47
## 21	21	-130.94
## 22	22	-142.71
## 23	23	-166.09
## 24	24	-163.94
## 25	25	-120.30
## 26	26	-153.58
## 27	27	-160.48
## 28	28	-164.78
## 29	29	19879.73
## 30	30	-100.69
## 31	31	-161.18
## 32	32	160.07
## 33	33	-166.31
## 34	34	-168.66
## 35	35	-159.02
## 36	36	-167.48
## 37	37	-166.94
## 38	38	-159.32
## 39	39	-167.27
## 40	40	-161.95
## 41	41	-169.53
## 42	42	-166.59
## 43	43	-109.47
## 44	44	-164.64
## 45	45	-154.33
## 46	46	-107.84
## 47	47	-169.67
## 48	48	-170.65
## 49	49	80.95
## 50	50	-161.37
## 51	51	-116.48
## 52	52	-166.57
## 53	53	-165.56
## 54	54	-119.84
## 55	55	-158.97
## 56	56	-157.14
## 57	57	-163.61
## 58	58	-161.94
## 59	59	-168.48
## 60	60	-167.23
## 61	61	-161.35
## 62	62	196.84
## 63	63	-166.33
## 64	64	-167.46
## 65	65	-164.17
## 66	66	-167.51
## 67	67	-167.63
## 68	68	-163.42
## 69	69	-108.43
## 70	70	-169.87

```
## 71      71    -156.88
## 72      72    -101.56
## 73      73    -165.99
## 74      74    -166.10
## 75      75    -145.22
## 76      76    -170.03
## 77      77    -166.63
## 78      78     275.26
## 79      79    -168.76
## 80      80    -170.63
## 81      81    -132.02
## 82      82    -168.28
## 83      83    -161.71
## 84      84    -136.88
## 85      85    -167.50
## 86      86    -159.82
## 87      87    -166.75
## 88      88    -144.02
## 89      89    -169.22
## 90      90     -41.07
## 91      91    -161.20
## 92      92    -107.72
## 93      93    -169.19
## 94      94    -126.08
## 95      95    -170.26
## 96      96    -167.03
## 97      97    -143.95
## 98      98    -170.33
## 99      99    -145.59
## 100     100    -157.00
```

```
sum(beneficios)
```

```
## [1] 6025.11
```

Finalmente, se podría haber considerado utilizar los precios de cualquier otra fecha para observar si varían los beneficios. Sin embargo, independientemente de la media de precios del mercado las probabilidades van a seguir siendo las mismas para las skins más comunes y más raras. Por lo tanto, van a seguir habiendo mayores o menores perdidas en general. Efectivamente, la afirmación de que nunca ganas dinero abriendo cajas se confirma.

Como curiosidad, podemos calcular la probabilidad de que nos toque el mejor caso posible, un ítem especial con StatTrak y que esté en estado Factory New. Como todos los sucesos son independientes multiplicamos las probabilidades:

```
(probs %>% pull(prob))[match("Special Item", probs$category)] * 0.1 * 0.07 * 100 # Probabilidad de Spec
```

```
## [1] 0.001792
```

Es decir, la probabilidad es de un 0.001792%.

Conclusiones

El mercado presenta una elevada actividad y liquidez global, con una gran mayoría de ítems participando en las transacciones. Sin embargo, el valor económico se encuentra concentrado en un número reducido de ítems de alto precio (+20% del valor del mercado a 2024), mientras que el precio mediano permanece estable. La relación negativa entre precio y liquidez confirma una estructura competitiva, y la distribución bimodal de la iliquidez sugiere la coexistencia de ítems muy activos con una cola de productos prácticamente inactivos. Y sin comentar que el mercado sufre muchas variaciones debido a actores de mala fe como las granjas de bots y por nuevo contenido cambiando la distribución del mercado.

Este está fuertemente a las manos de inversores que usan bots para comprar y vender armas a su vez que jugadores que invierten una cantidad considerable en la parte más popular del juego que es la apuesta por cajas que como hemos visto siempre perdemos a la larga. Sabiendo que niños juegan este juego trae preguntas éticas de hasta que punto este tipo de apuestas es correcto para exponer a jóvenes y como estas “élites” pueden manipularlos.

Honestamente, entramos a este estudio pensando encontrar estadísticas de un mercado agresivo, fuertemente dictado por grandes compradores y una pérdida para el usuario promedio. Pero terminamos encontrando como las condiciones de un país fuerzan individuos a lanzar bots para lograr ganar dinero en mercados controlados por bots y optimizado para ganar lo máximo posible de el.

Con esto quiero decir que esperábamos que CSGO fuera el “antagonista” que creara un mercado para beneficiarse de su audiencia, pero al contrario encontramos que la audiencia en si era la que se peleaba entre ella para lograr beneficios.

Esto es un tema poco estudiado, pero claramente en este tipo de economías se puede observar como se crea un mercado como el de la bolsa en periodos mas cortos y podrían ser fuentes para futuros economistas para poder estudiar estos fenómenos en más detalle.